CELLULAR AUTOMATON

Pau Fonseca i Casas, pau@fib.upc.edu

Key message

- IF you know how to formalize a model using SDL you know:
  - How to model ODEs
  - How to model MAS
  - How to model distributed simulation
  - How to integrate IoT
  - How to model CA

- So at the end we will follow SDL syntax in this introduction to CA.

Other alternatives?

- Yes
  - Wolfram rules
  - CELL-DEVS
  - Mathematical representation

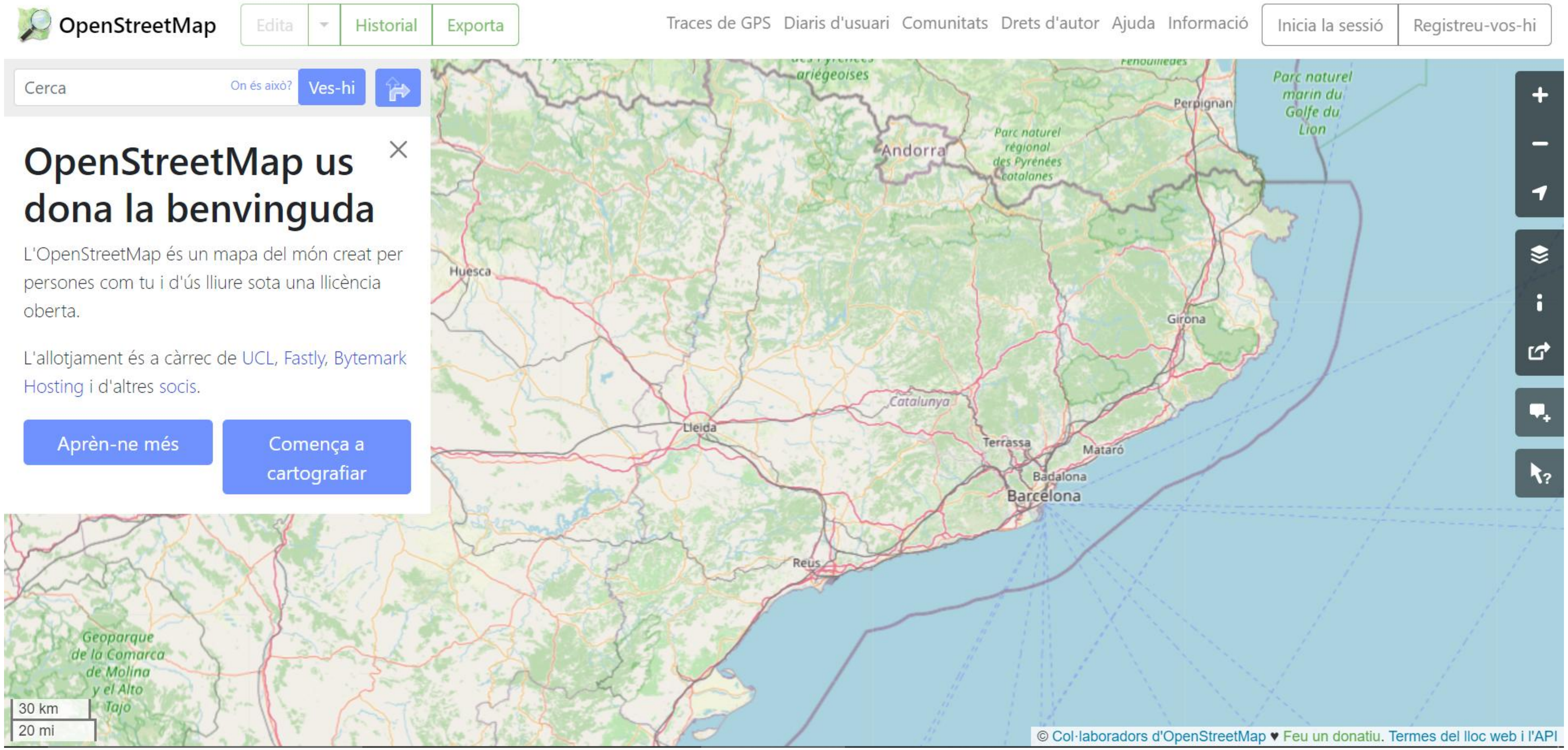
GIS data

GIS data structure and classification.

GIS data

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Open Street map



Layers in a GIS

Layer Type	Description	Examples
Vector	Points, lines, polygons	Roads, cities, land parcels
Raster	Grid of cells/pixels	Satellite imagery, DEMs
Attribute	Non-spatial data linked to spatial features	Population data, soil properties
Thematic	Specific themes (e.g., population density)	Choropleth maps, heatmaps
Base	Background/reference layers	OpenStreetMap, Google Maps
Network	Interconnected linear features	Road networks, utility networks
Temporal	Data that changes over time	Historical land use, weather patterns
3D	Three-dimensional features	Buildings, terrain
Annotation	Text or labels for features	City names, road labels
Surface	Continuous surfaces (e.g., elevation)	Contour lines, TIN
Geodatabase	Structured collection of geographic data	Feature classes, raster datasets
WMS/WFS	Web-based layers	Online maps, shared datasets
CAD	Imported from CAD systems	Engineering designs
LiDAR	3D point cloud data	Flood modeling, forestry
Statistical	Statistical data represented spatially	Census data, disease incidence

Layers in a GIS

Layer	Description
DEM (raster layers)	Raster, thematic, base, temporal, geodatabase, WMS/WFS
2DLayers (vector layers)	Vector, annotation, surface, WMS/WFS, Statistical
3DLayers (3D features)	3D, WMS/WFS
Routes (GPS, IoT)	Track points, WMS/WFS
2DObjects (attribute layers)	Attribute, WMS/WFS
3DObjects (attribute layers)	WMS/WFS, CAD, LiDAR

Cellular automata

Modeling the environment

Cellular Automaton (CA) definition

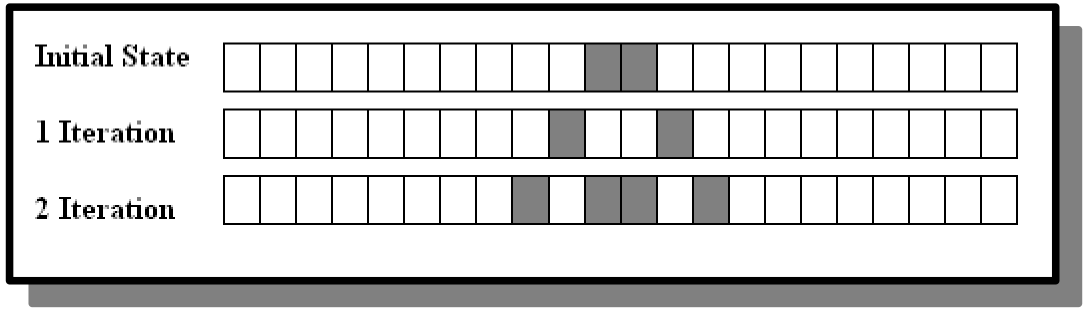
- Cellular Automaton (CA) consists of a regular grid of cells, each in one of a finite number of states, such as on and off (in contrast to a coupled map lattice). The grid can be in any finite number of dimensions.

Cellular Automaton (CA) definition

- For each cell, a set of cells called its neighborhood is defined relative to the specified cell. An initial state (time $t = 0$) is selected by assigning a state for each cell. A new generation is created (advancing t by 1), according to some fixed rule (generally, a mathematical function) that determines the new state of each cell in terms of the current state of the cell and the states of the cells in its neighborhood.
  - Source: https://en.wikipedia.org/wiki/Cellular_automaton

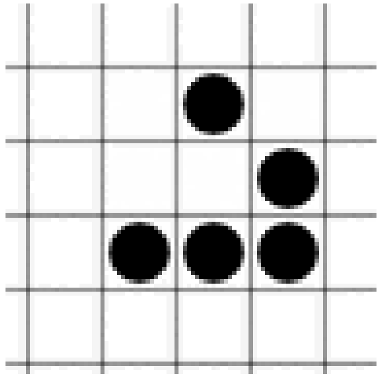
Cellular automaton

- Structure based in:
  - Set of rules.
  - Matrix of data.
- Modify the matrix following the set of rules.



Game of life

- The **Game of Life** is a cellular automaton devised by the British mathematician John Horton Conway in 1970. It is the best-known example of a cellular automaton.
- Glider gun and glider



UNIVERSITAT POLITÈCNICA DE CATALUNYA

Cellular automaton

- Simplifies the use of environment in a simulation model.
  - Torrens, Paul M. Benenson, Itzhak. 2004, Geosimulation, John Wiley & Sons Ltd., ISBN: 0-470-84349-7
  - Batty, Michael, 2005, Cities and complexity.The MIT Press.
  - Wolfram, Stephen 2002, A new Kind of Science, Wolfram Media, Inc.

Cellular automaton

- Considerations:
  - The space of states can be huge.
  - Not all the states can be reachable → maps of states space.
- Limitations:
  - Only one matrix of data.
  - Only one set of rules.
  - The space is discrete.

Basics of One-Dimensional Cellular Automata

- A one-dimensional cellular automaton consists of a line of cells, where each cell can be in one of two states: 0 (off/dead) or 1 (on/alive).
- The state of each cell in the next time step depends on its current state and that of its immediate neighbors (typically, the left and right neighbors).
- The evolution rule defines how the state of a cell changes based on the states of its neighborhood.

Neighborhood and Configurations

- In a one-dimensional cellular automaton with a neighborhood size of 1 (i.e., each cell has one neighbor to the left and one to the right), there are 8 possible configurations for the neighborhood of 3 cells (left, center, right):
  - 111, 110, 101, 100, 011, 010, 001, 000

- Each configuration is associated with a new state for the center cell in the next time step.

Wolfram Encoding Rule

- The Wolfram rule assigns an 8-bit binary number to each one-dimensional cellular automaton, where each bit represents the new state of the center cell for one of the 8 possible configurations.
- The binary number is converted to a decimal number between 0 and 255, which identifies the rule.

Example

- Suppose the rule assigns the following new states:
  - 111 → 0
  - 110 → 1
  - 101 → 1
  - 100 → 0
  - 011 → 1
  - 010 → 1
  - 001 → 1
  - 000 → 0
- The corresponding binary number is: 01101110. Converted to decimal: **110.**

- Therefore, this rule is called Rule **110.**

Notable Rules

- Rule 30: Generates chaotic patterns and is known for its complex behavior.
- Rule 90: Produces fractal patterns and follows the logic of the Sierpinski triangle.
- Rule 110: It is Turing-complete, meaning it can perform any computational calculation given the appropriate initial state.

- Rule 184: Used to model traffic flow.

Visualization of the Rules

Tools

- Wolfram rules can be visualized as transition tables or evolution diagrams:
  - Transition table: Shows the 8 neighborhood configurations and the corresponding new state.
  - Evolution diagram: Shows how the automaton evolves over time, row by row.

Example of a transition table for Rule 110

Neighborhood (left, center, right)	New State
1 1 1	0
1 1 0	1
1 0 1	1
1 0 0	0
0 1 1	1
0 1 0	1
0 0 1	1
0 0 0	0

Elementary Cellular Automata

$$A = \left(a(t), S_A, f_A\right)$$

lattice of cells — alphabet — update function

	$a_{n-1}(t)$	$a_n(t)$	$a_{n+1}(t)$	
		$a_n(t+1)$		

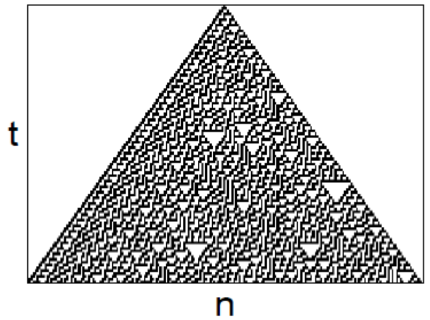
$$S_A = \{0,1\}$$

for elementary CA

$$a_n(t+1) =$$
$$f_A\left[a_{n-1}(t), a_n(t), a_{n+1}(t)\right]$$

(256 possible functions)

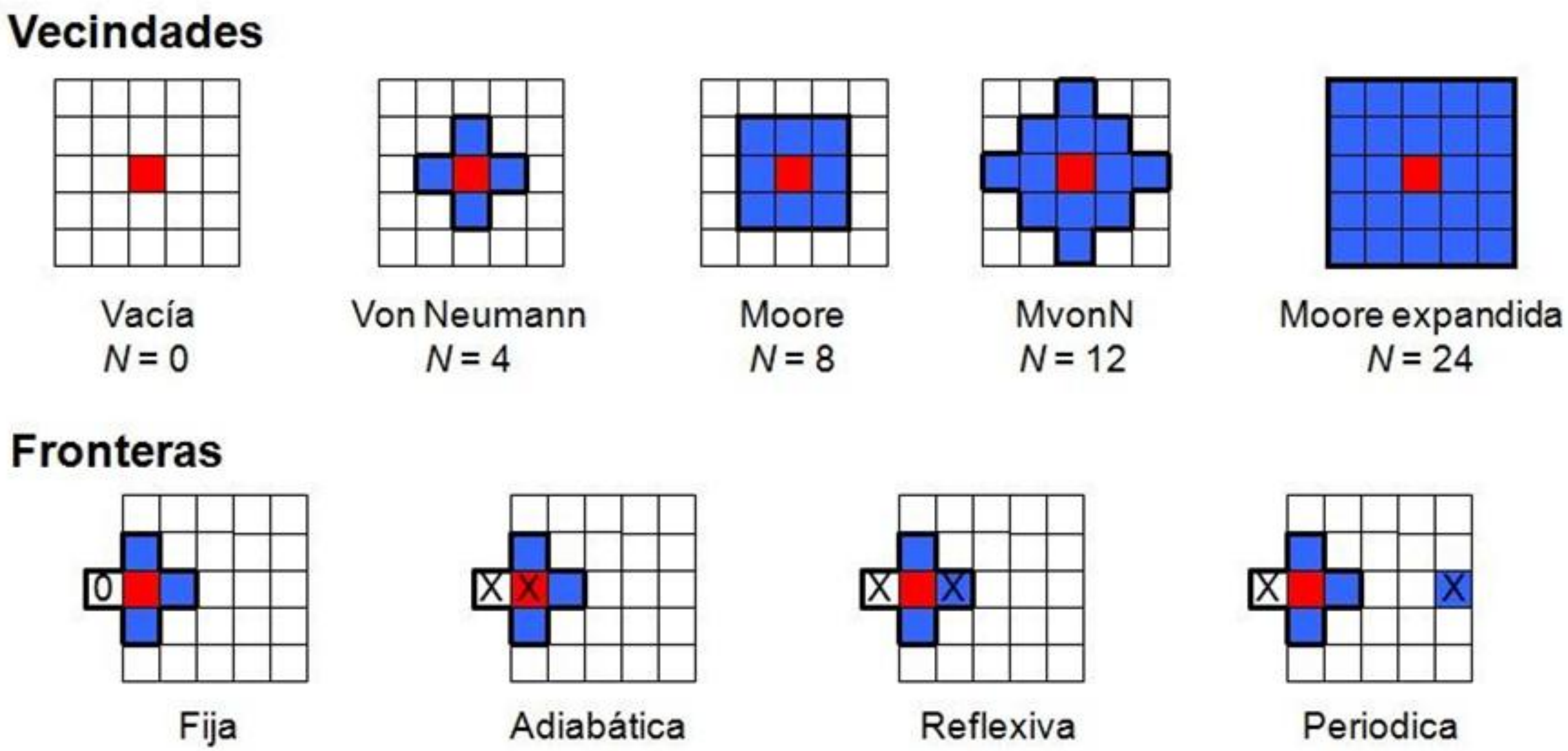
A dynamical system capable of rich behavior.



Wolfram's rule notation:
read
as a binary number.

rule 30	0	0	0	1	1	1	1	0
rule 54	0	0	1	1	0	1	1	0
rule 60	0	0	1	1	1	1	0	0
rule 62	0	0	1	1	1	1	1	0
rule 90	0	1	0	1	1	0	1	0
rule 94	0	1	0	1	1	1	1	0
rule 126	0	1	1	1	1	1	1	0
rule 150	1	0	0	1	0	1	1	0
rule 158	1	0	0	1	1	1	1	0
rule 182	1	0	1	1	0	1	1	0
rule 188	1	0	1	1	1	1	0	0
rule 190	1	0	1	1	1	1	1	0

Neighbourhood



Source: https://centrobio.utec.edu.pe/automatas-celulares-una-tecnica-inteligente-para-modelar-sistemas-complejos/

Game of life

- The Environment
  - The Game of Life takes place on a two-dimensional grid of cells, typically square.
  - Each cell can be in one of two states: alive (1) or dead (0).
  - The state of each cell evolves in discrete time steps (called generations) based on its neighbors.
- Neighborhood: Moore

Game of life: Evolution Rules

- Birth:
  - A dead cell comes to life if it has exactly 3 live neighbors.
- Survival:
  - A live cell survives if it has 2 or 3 live neighbors.
  - If it has fewer than 2 live neighbors, it dies due to underpopulation.
  - If it has more than 3 live neighbors, it dies due to overpopulation.

CA classification

Differents ways to classify CA

Basic criteria for the classification

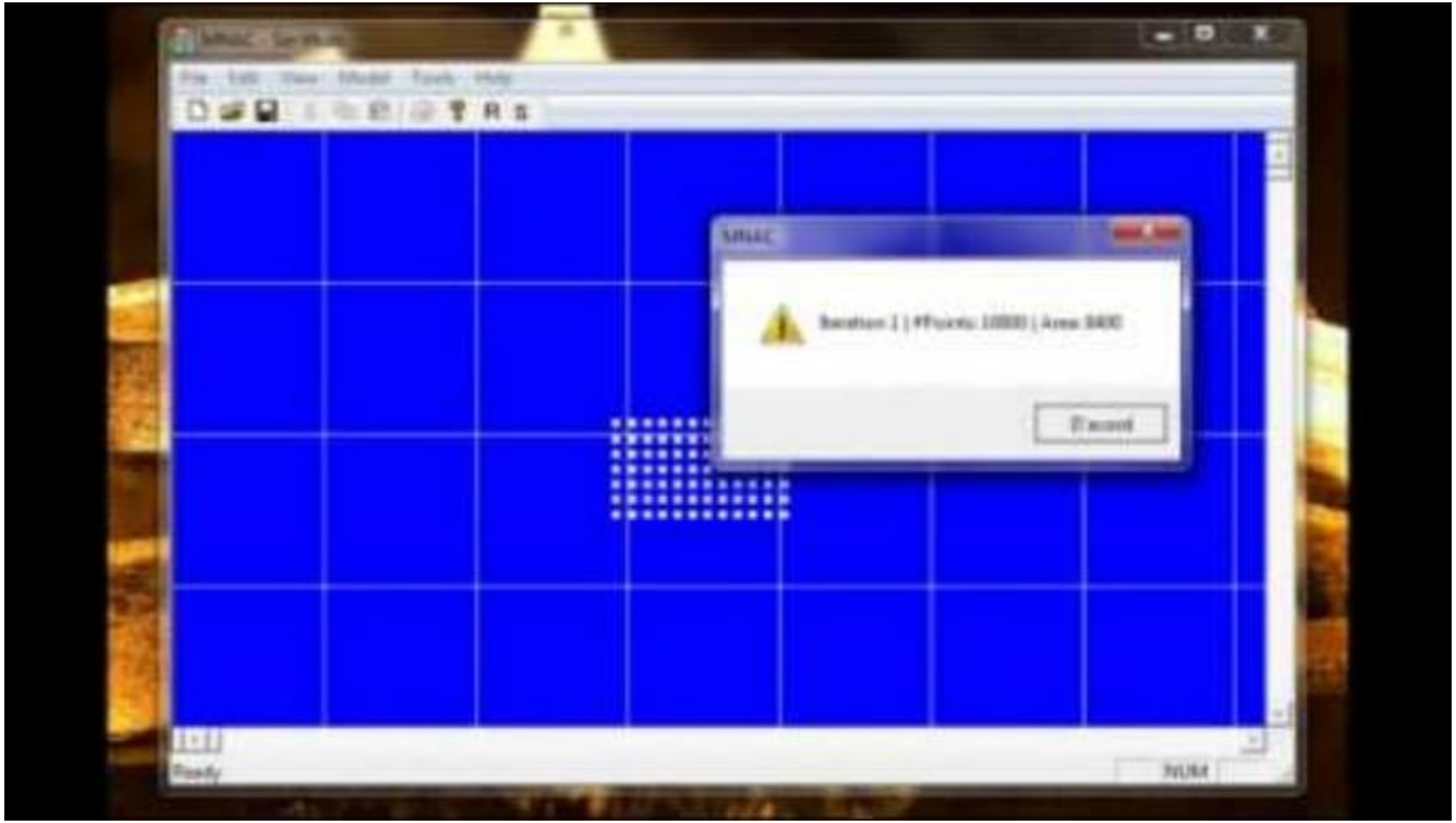
- Dimension
- Neighborhood structure
- Rule type
- Behaviour (Wolfram)

Classification by Dimensionality

- One-Dimensional (1D) Cellular Automata:
  - Cells are arranged in a linear array.
  - Example: Rule 30, Rule 110.
- Two-Dimensional (2D) Cellular Automata:
  - Cells are arranged in a grid (e.g., square, hexagonal, or triangular).
  - Example: Conway's Game of Life.
- Three-Dimensional (3D) Cellular Automata:
  - Cells are arranged in a 3D lattice.
  - Example: 3D extensions of Conway's Game of Life.

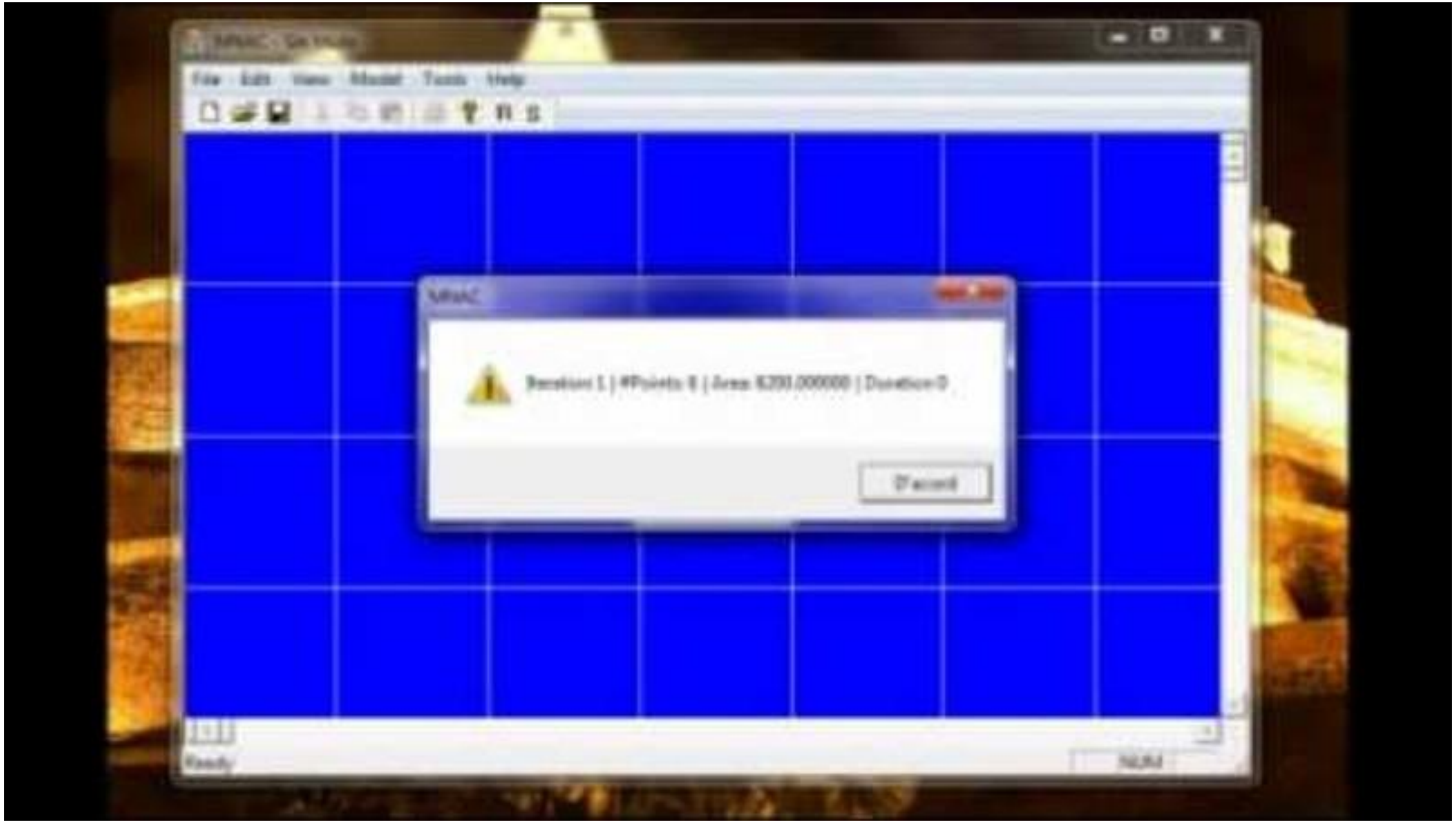
Classification by Neighborhood Structure

- Von Neumann Neighborhood:
  - Includes the four orthogonally adjacent cells (up, down, left, right).
  - Commonly used in 2D cellular automata.
- Moore Neighborhood:
  - Includes all eight surrounding cells (orthogonal and diagonal).
  - Commonly used in 2D cellular automata.
- Extended Neighborhoods:
  - Larger neighborhoods, such as radius-2 or radius-3 neighborhoods.
- Topological Neighborhood:
  - Neighborhoods defined using functions over a topological space, accepts changes, etc. like m:n-CA^{k}.



https://youtu.be/49OoqWE0REs?si=1ygpWmCYsk6MLcsc

m:n-CA^k



https://youtu.be/PiXiq7j9A2s?si=hBVyCir3mZe09_ns

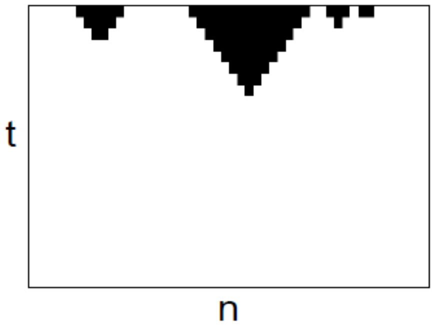
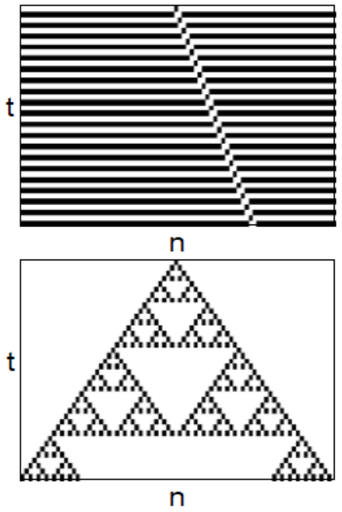
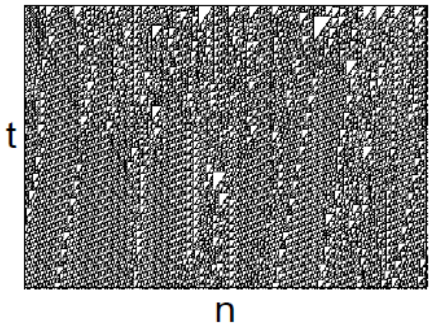
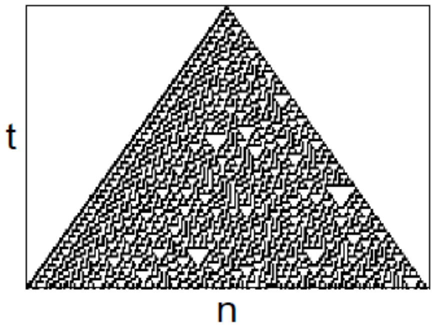
Classification by Rule Type

- Totalistic Rules:
  - The new state of a cell depends on the sum of the states in its neighborhood.
  - Example: Brian's Brain (a 2D cellular automaton).
- Outer-Totalistic Rules:
  - The new state depends on the current state of the cell and the sum of the states in its neighborhood.
  - Example: Conway's Game of Life.
- Non-Totalistic Rules:
  - The new state depends on the specific configuration of the neighborhood, not just the sum.
  - Example: Rule 110 (1D cellular automaton).

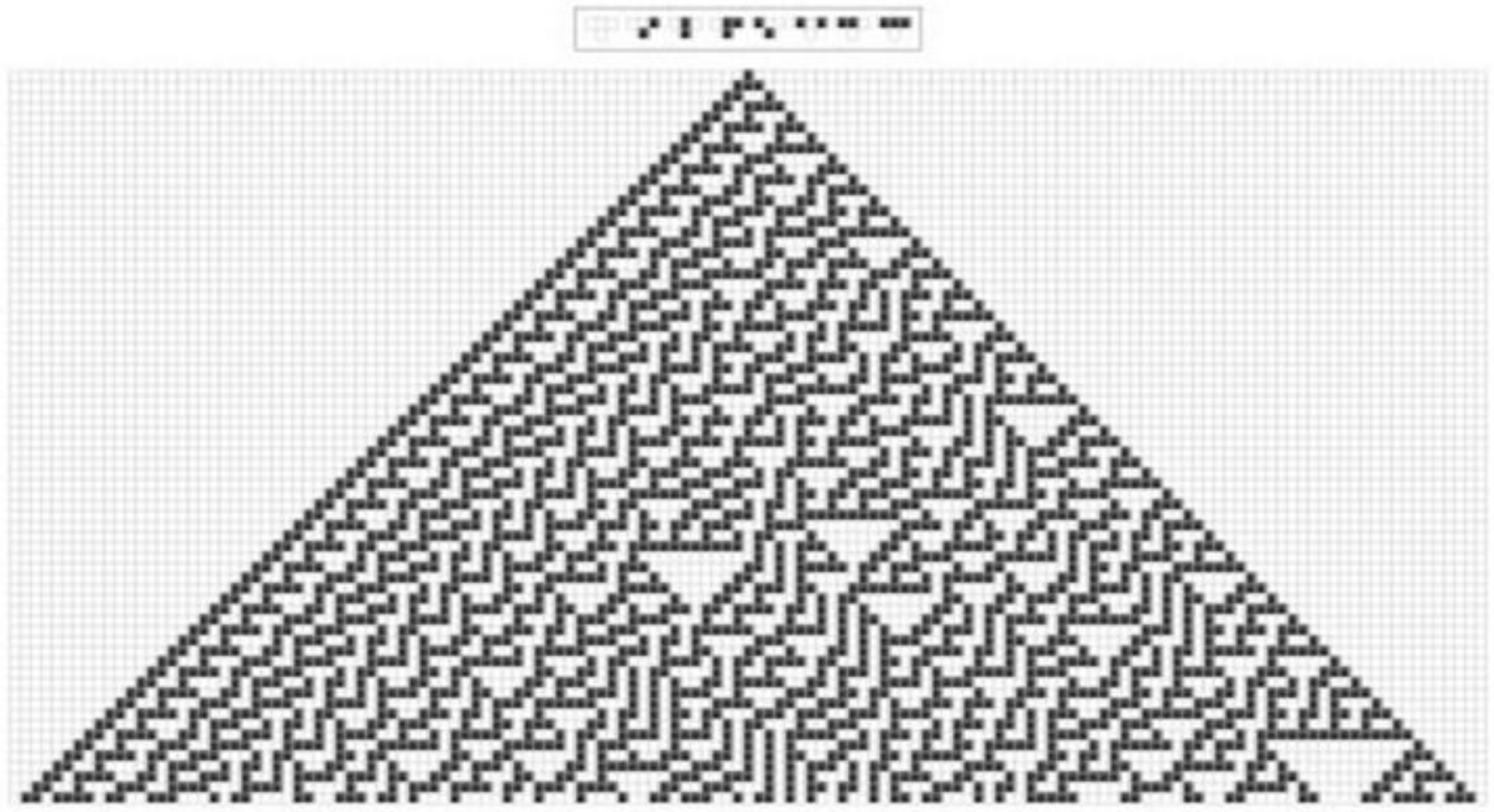
Classification by Behavior (Wolfram's Classes)

- Stephen Wolfram proposed a classification of cellular automata based on their long-term behavior:
- Class I: Homogeneity:
  - The system evolves to a uniform state (all cells have the same value).
  - Example: Rule 0 (all cells become 0).
- Class II: Periodicity:
  - The system evolves to a periodic or stable pattern.
  - Example: Rule 4 (repeats a simple pattern).
- Class III: Chaos:
  - The system evolves to a chaotic, aperiodic pattern.
  - Example: Rule 30 (produces complex, unpredictable behavior).
- Class IV: Complexity:
  - The system exhibits complex, localized structures that interact in

Wolfram’s classification of CA

<u>**Class 1**</u> **Decaying structures**	<u>**Class 2**</u> **Periodic or nested**	<u>**Class 4**</u> **Interacting structures**	<u>**Class 3**</u> **Chaotic behavior**
			

Rule 30



https://www.youtube.com/watch?v=7izmHXpcc5A

Wolfram's classification of CA

- Classification is only qualitative, there is no classification algorithm.
- CA seem to cover the full range of complexity found in nature.
- Some CA are **computationally irreducible** (Wolfram hypothesized that most of class 3 and 4).
- Some CA are **universal Turing Machines** (Wolfram hypothesized that class 4 is the threshold).

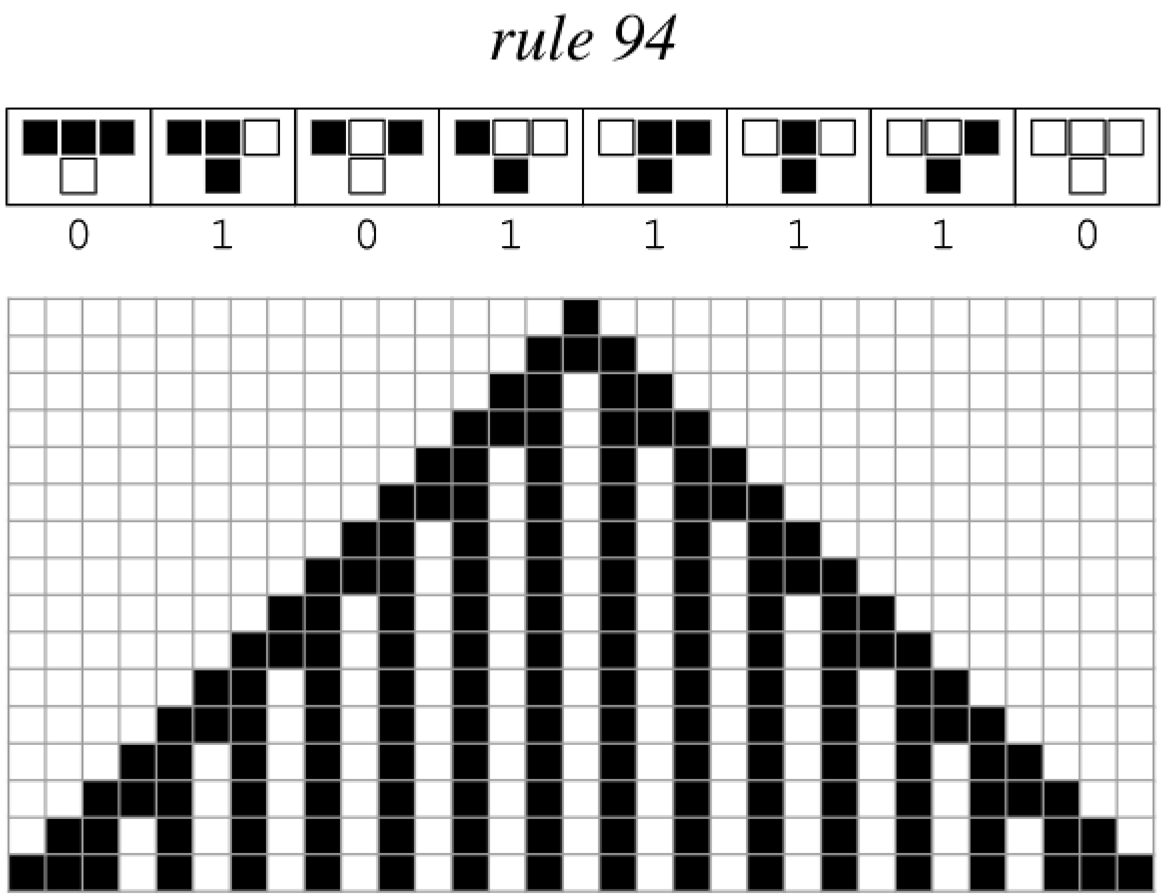
Computational Irreducibility and Predictability

- Computational irreducibility = the system behavior can be predicted only by running it. There is no computationally more efficient way.
- Some complex CA are computationally irreducible.
  - Wolfram hypothesized that most of class 3 and 4 CA are irreducible.
- In analogy to the real world:
  - Many physical processes that seem complex are probably computationally irreducible.
  - Most physical systems have too many degrees of freedom for direct simulation.
  - Are Complex Systems inherently unpredictable?

Working with them

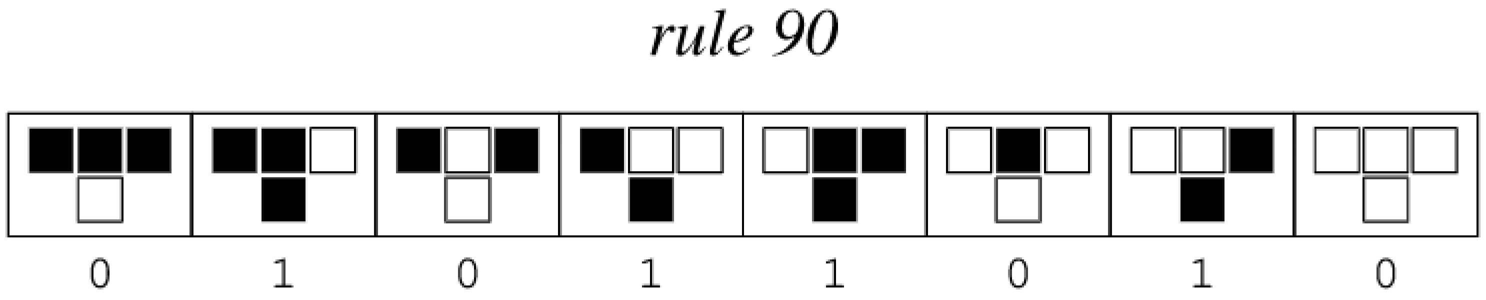
Some examples

Rule 94

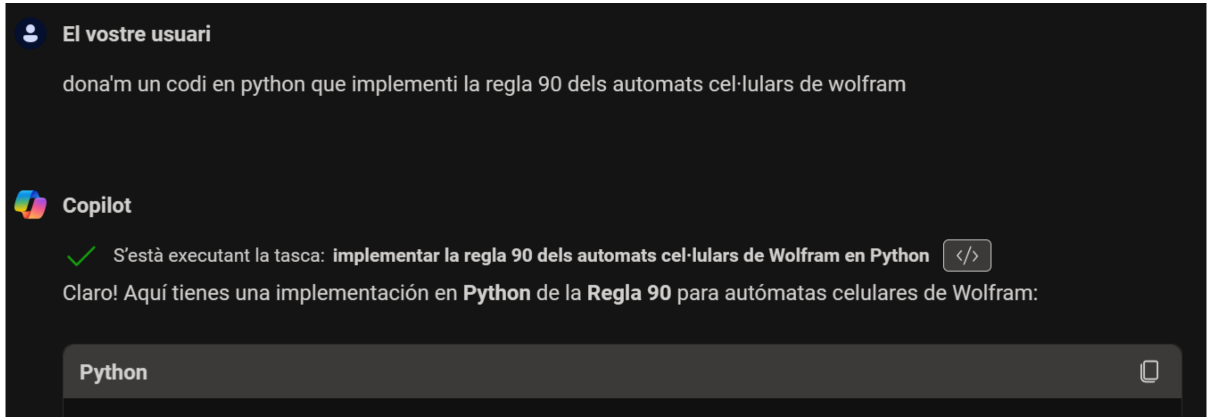


Source:: Weisstein, Eric W. "Rule 94." From *MathWorld*--A Wolfram Web Resource. https://mathworld.wolfram.com/Rule94.html

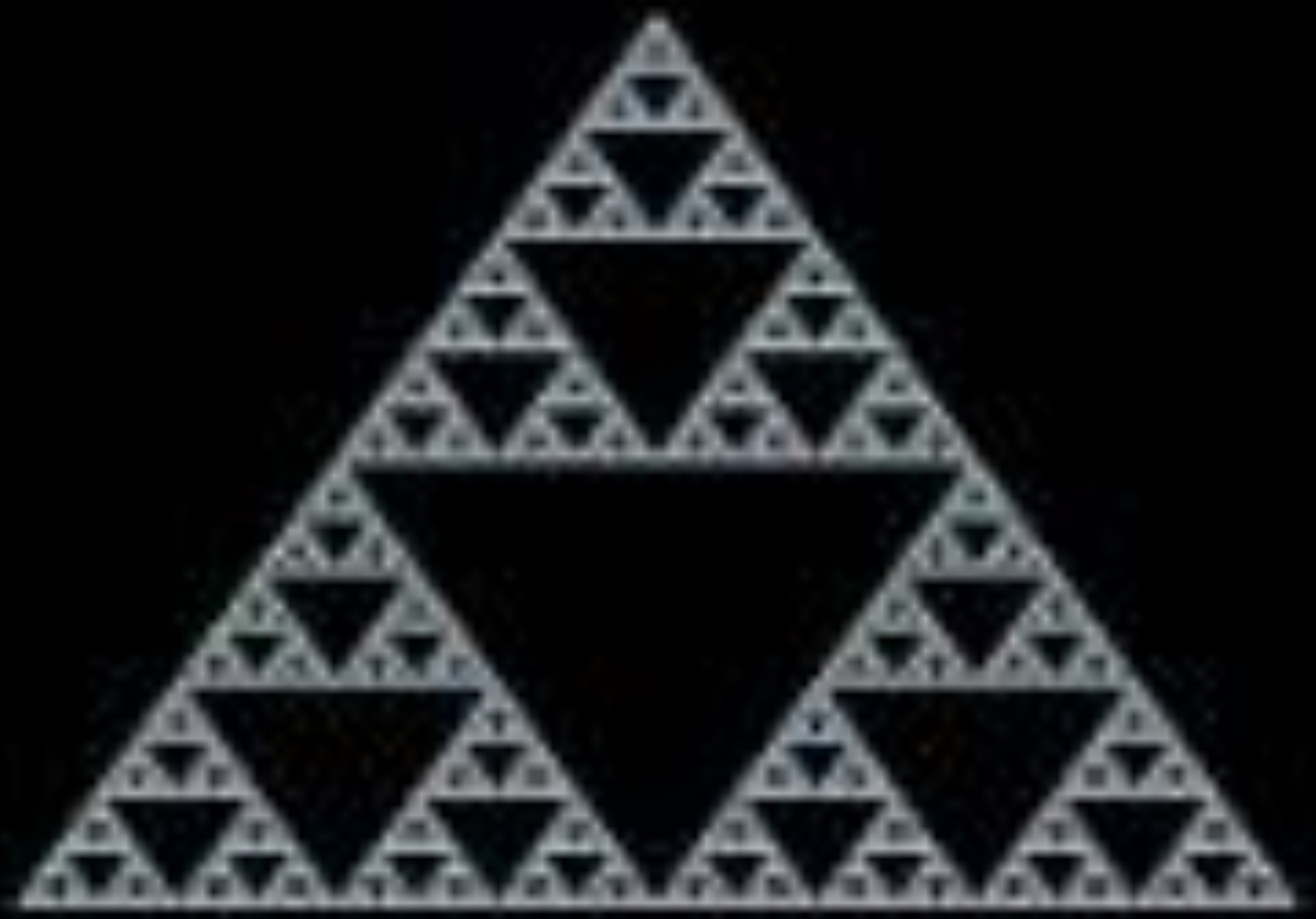
Rule 90



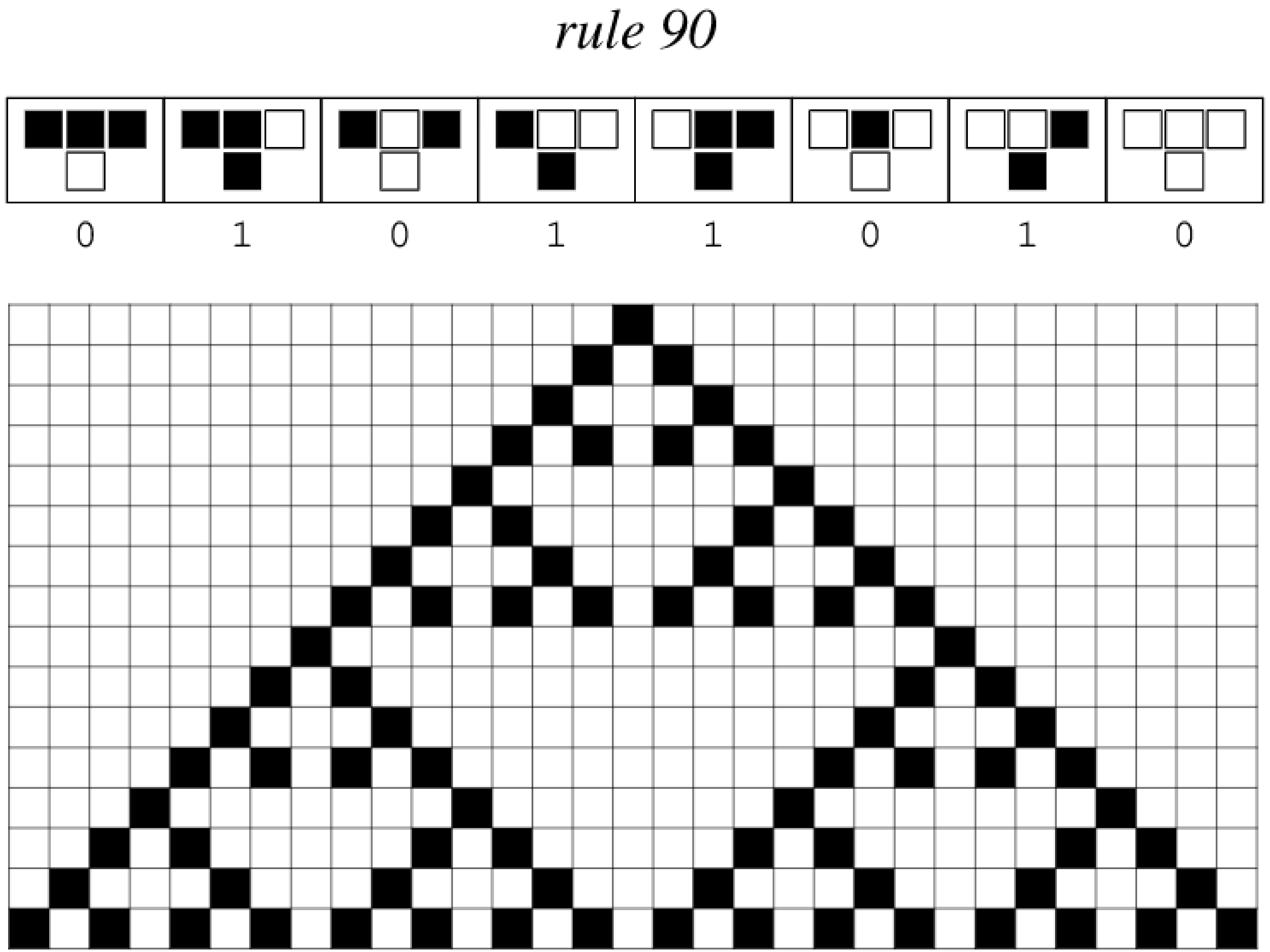
```python
def regla_90(pasos):
    # Inicializamos la primera fila con una sola célula negra en el centro
    fila = [0] * pasos
    fila[pasos // 2] = 1

    # Inicializamos la lista de resultados con la primera fila
    resultado = [fila]

    # Generamos filas posteriores basadas en la regla 90
    for _ in range(pasos - 1):
        nueva_fila = []
        for i in range(pasos):
            izquierda = fila[i - 1] if i > 0 else 0
            centro = fila[i]
            derecha = fila[i + 1] if i < pasos - 1 else 0
            nueva_celula = (izquierda + centro + derecha) % 2
            nueva_fila.append(nueva_celula)
        fila = nueva_fila
        resultado.append(fila)

    return resultado

# Establecemos el número de pasos (filas) para el autómata
pasos = 20

# Generamos el autómata con la regla 90
automata = regla_90(pasos)

# Imprimimos el autómata
for fila in automata:
    print("".join(str(celula) for celula in fila))
```



UNIVERSITAT POLITÈCNICA DE CATALUNYA



Rule 90

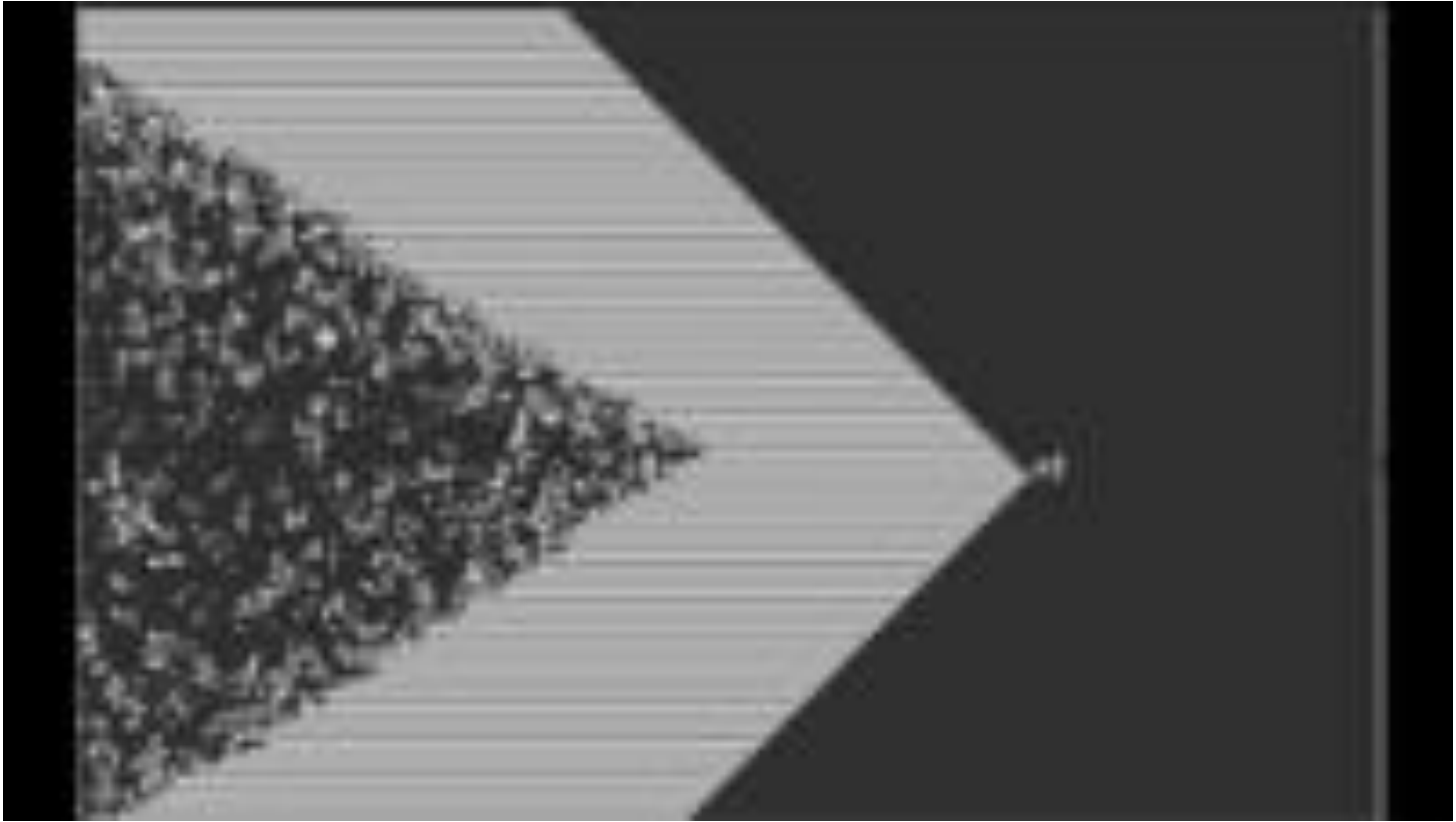


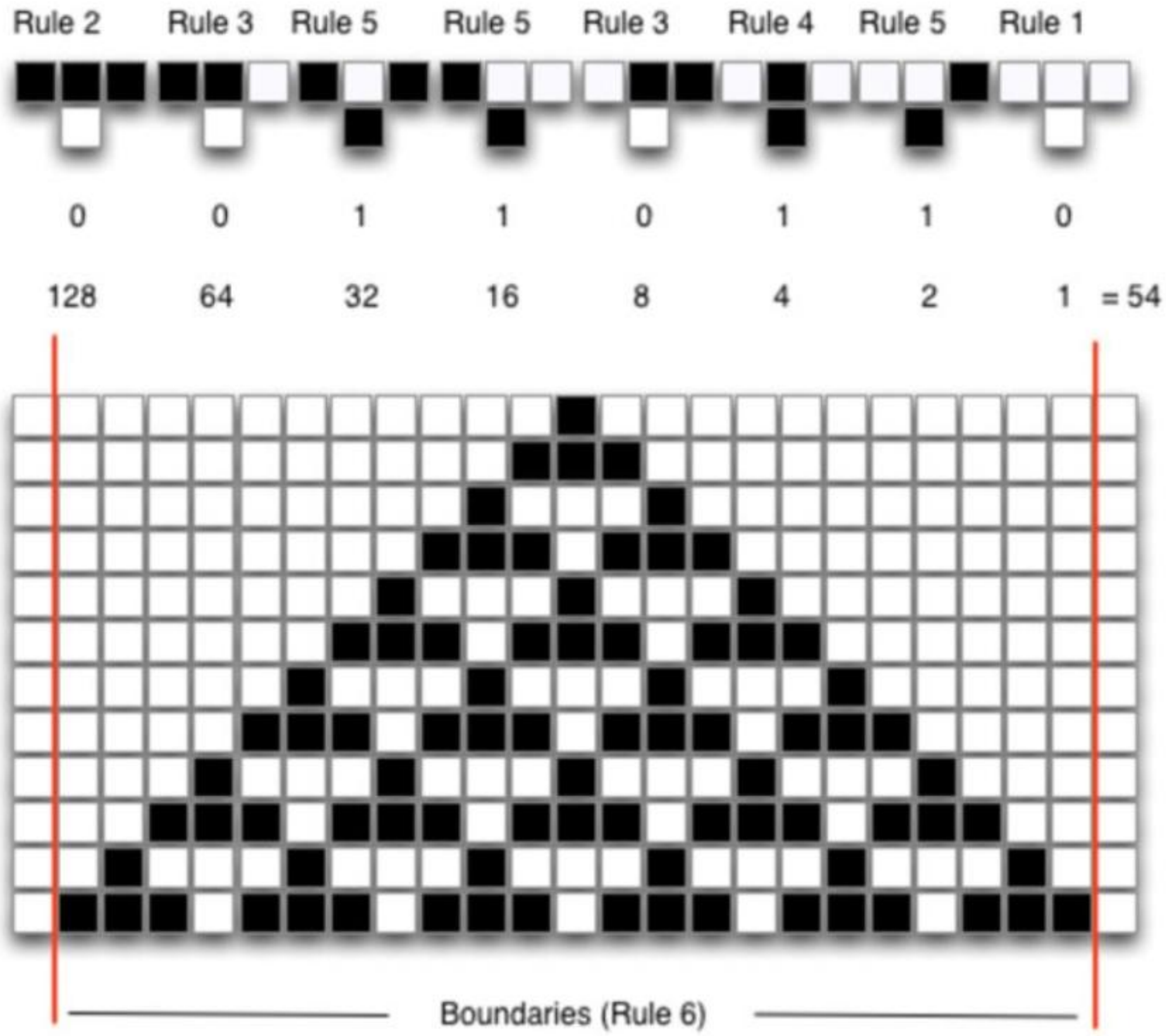
Source: Weisstein, Eric W. "Rule 90." From *MathWorld*--A Wolfram Web Resource. https://mathworld.wolfram.com/Rule90.html

Rule 54 Cellular Automaton

- Rule 54 Cellular Automaton

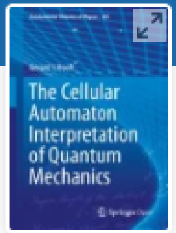
Source: https://www.researchgate.net/publication/317311801_Cellular_Automata_as_an_Example_for_Advanced_Beginners%27_Level_Coding_Exercises_in_a_MOOC_on_Test_Driven_Development/figures?lo=1




Rule 2
Rule 3
Rule 5
Rule 5
Rule 3
Rule 4
Rule 5
Rule 1
0 0 1 1 0 1 1 0
128 64 32 16 8 4 2 1 = 54
Boundaries (Rule 6)

m:n-CA^k

Extending the capabilities of the cellular automata



Book | Open Access | © 2016

The Cellular Automaton Interpretation of Quantum Mechanics

Home > Book

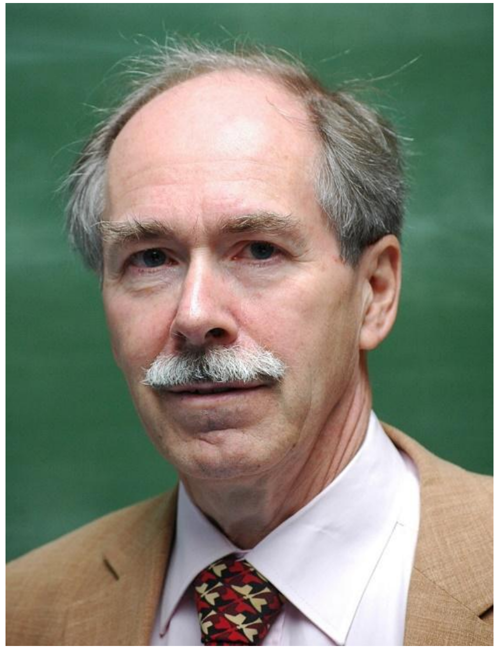
Authors: Gerard 't Hooft

A radically different approach to the challenges of quantum theory

Authored by one of today's most creative theoretical physicists

Pedagogically structured to first introduce concepts and main arguments, with technical details presented separately

Part of the book series: Fundamental Theories of Physics (FTPH, volume 185)



[The Cellular Automaton Interpretation of Quantum Mechanics | SpringerLink]

New!

Complexity

Forward Series

Research Article | Open Access

The Multi-n-Dimensional Cellular Automaton: A Unified Framework for Tensorial, Discrete, and Continuous Simulations—A Computational Definition of Time

Pau Fonseca i Casas

First published: 26 November 2025 | https://doi.org/10.1155/cplx/3088010 | VIEW METRICS

Academic Editor: Pramita Mishra

https://onlinelibrary.wiley.com/doi/full/10.1155/cplx/3088010

What is Topology?

- Definition: Topology is a branch of mathematics that studies properties of space that are preserved under continuous deformations (stretching, bending, twisting, but not tearing or gluing).
- Key Idea: Focuses on the concept of "closeness" and "continuity" without relying on distance.
- Visual: Show a coffee cup transforming into a donut (classic example of topological equivalence).

Key Concepts in Topology

- Topological Space: A set of points with a collection of open sets satisfying certain axioms.
- Continuity: A function is continuous if the inverse image of an open set is open.
- Homeomorphism: A bijective continuous function with a continuous inverse (preserves topological properties).

Topological invariants

- A topological invariant is a property of a topological space that is preserved under homeomorphisms (continuous deformations). If two spaces have different invariants, they cannot be homeomorphic.

Connectedness

- A topological space is connected if it cannot be divided into two disjoint non-empty open sets. Intuitively, it is "all in one piece."

Types of Connectedness:

- Connected:
  - A space is connected if it is not the union of two disjoint non-empty open sets.
  - Example: The interval [0,1] is connected.
- Path-Connected:
  - A space is path-connected if any two points can be connected by a continuous path.
  - Example: A circle is path-connected.
- Simply Connected:
  - A space is simply connected if it is path-connected and every loop can be continuously shrunk to a point.
  - Example: A sphere is simply connected, while a torus is not.
- Disconnected:
  - A space is disconnected if it can be divided into two disjoint non-empty open sets.
  - Example: The set [0,1)U(1,2] is disconnected.

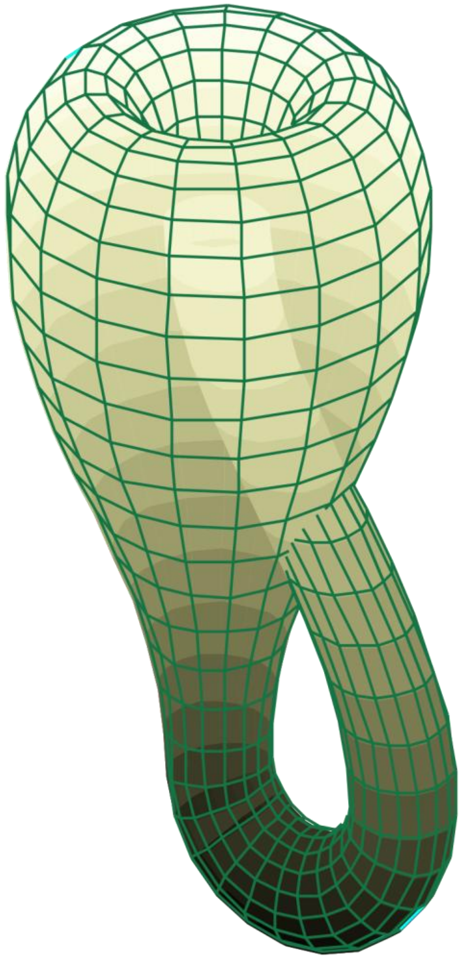
Examples of Topological Spaces

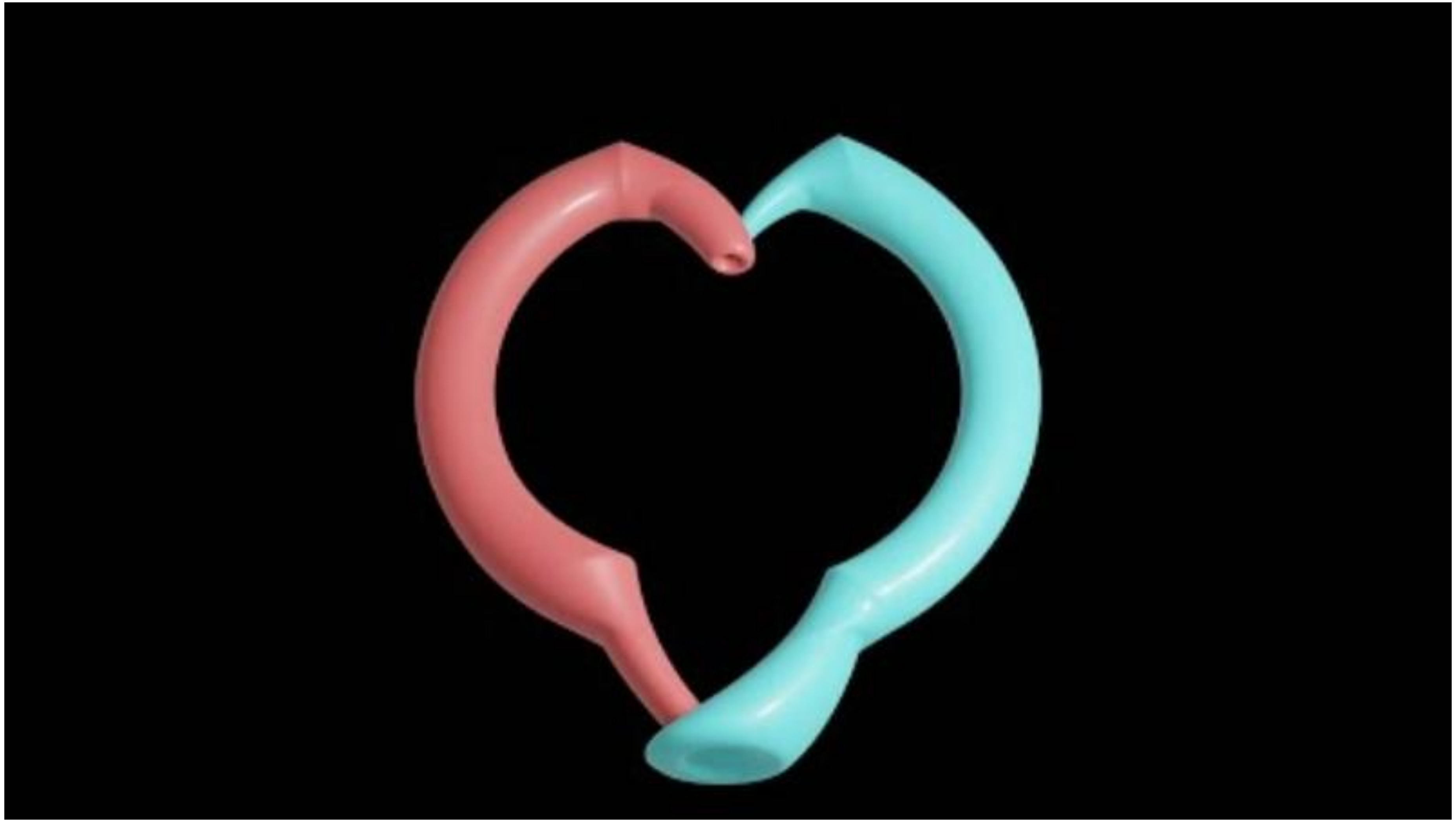
- Euclidean Space ($\mathbb{R}^n$): The standard space with the usual topology.
- Sphere (S^2): The surface of a ball.
- Torus (T^2): The surface of a donut.
- Möbius Strip: A surface with only one side and one edge.

Topological Equivalence

- Homeomorphism: Two spaces are topologically equivalent if there is a homeomorphism between them.
  - A coffee cup and a donut are homeomorphic.
  - A circle and a square are homeomorphic.
- Non-Examples:
  - A circle and a line are not homeomorphic.

Torus Eversion: Klein Bottles





Definition of an Open Set

- In topology, an open set is a set that satisfies certain axioms, which are designed to generalize the intuitive notion of "openness" in familiar spaces like $\mathbb{R}^n$.
- The empty set and the whole set are open:
  - $\emptyset \in \tau$
  - $X \in \tau$
- Arbitrary unions of open sets are open:
  - If $\{U_i\}_{i \in I}$ is a collection of sets in τ, then $\bigcup_{i \in I} U_i \in \tau$.
- Finite intersections of open sets are open:
  - If $U_1, U_2, \dots, Un \in \tau$, then $U_1 \cap U_2 \cap \cdots \cap Un \in \tau$.
- The sets in τ are called open sets, and the pair (X,τ) is called a topological space.

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Intuitive Understanding of Open Sets

- In $\mathbb{R}^n$, an open set is a set where every point has a "neighborhood" entirely contained within the set.
- For example:
  - An open interval (a,b) in $\mathbb{R}$ is an open set.
  - An open ball in $\mathbb{R}^n$ (a disk without its boundary) is an open set.
- In topology, the concept of an open set is abstracted to any set X, ven if X does not have a notion of distance.

Examples of Topologies Generated by Open Sets

- Standard Topology on $\mathbb{R}$:
  - The open sets are unions of open intervals (a,b)(a,b).
  - Example: $(0,1) \cup (2,3)$ is an open set in $\mathbb{R}$.
- Discrete Topology:
  - Every subset of X is open.
  - Example: If X={a,b,c}X={a,b,c}, then $\tau=\{\emptyset,\{a\},\{b\},\{c\},\{a,b\},\{a,c\},\{b,c\},X\}$.
- Indiscrete Topology:
  - Only $\emptyset$ and X are open.
  - Example: If $X=\{a,b,c\}$, then $\tau=\{\emptyset,X\}$.
- Finite Complement Topology:
  - A set is open if its complement is finite or the whole space.
  - Example: On $\mathbb{R}$, the set $\mathbb{R} \setminus \{1,2,3\}$ is open.

Why Are Open Sets Important?

- An open set is a subset of a topological space that satisfies the axioms of a topology.
- Open sets are the building blocks of a topological space, defining its structure and enabling the study of continuity, convergence, and other topological properties.
- Different choices of open sets lead to different topologies, allowing for a wide range of mathematical exploration and application.
- Key properties
  - Flexibility: Open sets allow us to define a topology on any set, even if the set does not have a natural notion of distance.
  - Generalization: They generalize familiar concepts like continuity and convergence to abstract spaces.
  - Structure: They provide a way to describe the "shape" and "structure" of a space without relying on metrics.

Working with a topology

- Defining a topological space involves specifying a set X and a collection of subsets of X (called open sets) that satisfy certain axioms.
- These axioms generalize the intuitive notion of "openness" and provide a foundation for studying continuity, convergence, and other topological properties.

Multi n-dimensional CA

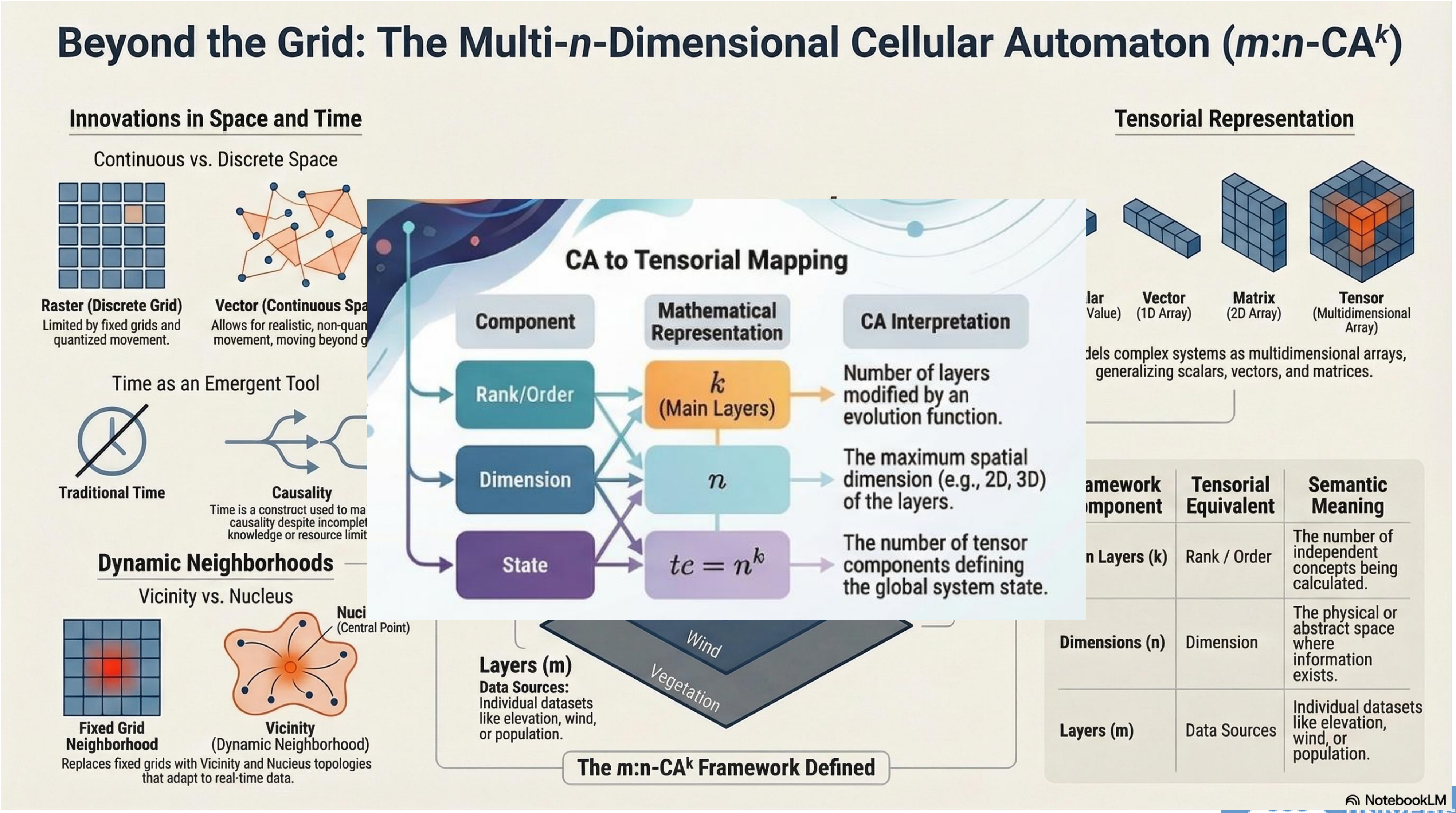
- A multi n dimensional cellular automaton is a cellular automaton generalization composed by m layers with n dimensions each one.
- Aim:
  - Allows multiple layers.
  - Allows vectorial layers (continuous space).
  - Allows multiple set of rules (evolution functions).

Multi n—dimensional CA

- The representation is: $m\!:\!n - AC^k$
- where
  - m: is the automaton number of layers.
  - n: is the different layers dimension.
  - k: is the number of main layers (1 by default).

For k=1, m=1 we have a common cellular automata.
For k=0 we have a dataset.

Multi n—dimensional CA



UNIVERSITAT POLITÈCNICA DE CATALUNYA

Multi n-dimensional CA

- Defined over the mathematical topology concept.
- $1{:}\,n - AC^{\,1}$ is the common cellular automata if the topology used is the discrete topology defined over $\mathbb{R}$ or $\mathbb{Z}$.
- The implementation, as is usual, can be a matrix.

State of the automata

- S_m^t state of the automaton, while E_m considering geo-references.
- Em[x1,..,xn], layer m state in x1,..,xn position
  - Em is a function describing cell state in position x1,..,xn of layer m.
- EG[x1,..,xn], automata status in x1,..,xn position.
  - EG returns automata global state in position referenced by coordinates x1,..,xn.

Evolution function Λm

- Evolution function allows global automaton state change through cells value modification.
- Defined for the layer m to modify its state through the state of others layers using combination function Ψ, and vicinity and nucleus functions.
- Is only defined in main layers.

Evolution function Λm

- Function defined for the layer m to modify its state through the state of others layers using combination function Ψ, and vicinity and nucleus functions.
- A m:n-AC automaton only presents one main layer, an m:n-ACk automaton presents k main layers.

Vicinity topology

- Topology defining the set of points (neighborhood) for layer m, to be considered for Λm calculus.
- Vicinity function vn(x1,..,xn):
  - Function returning minimum open set of vicinity topology containing point x1,..,xn, and including maximum points that accomplishes the restriction and minimum points not accomplishing the restriction.

Nucleous topology

- Topology defining the set of points (neighborhoods) for layer m, to be modified by Λm calculus
- Nucleus function nc(x1,..,xn)
  - Function returning minimum open set of nucleus topology containing point x1,..,xn, and including maximum points that accomplishes the restriction and minimum points not accomplishing the restriction.

Vicinity function example

- Over Z
  - vn(x1,..,xn) = {(x1-1,x2-1,xn-1), (x1-1,x2-1,Xn), (x1-1,x2-1,xn+1), (x1-1,x2,xn-1), (x1-1,x2,Xn), (x1-1,x2,xn+1), (x1-1,x2+1,xn-1), (x1-1,x2+1,Xn), (x1-1,x2+1,xn+1), (x1,x2-1,xn-1), (x1,x2-1,Xn), (x1,x2-1,xn+1), (x1,x2,xn-1), (x1,x2,Xn), (x1,x2,xn+1), (x1,x2+1,xn-1), (x1,x2+1,Xn), (x1,x2+1,xn+1), (x1+1,x2-1,xn-1), (x1+1,x2-1,Xn), (x1+1,x2-1,xn+1), (x1+1,x2,xn-1), (x1+1,x2,Xn), (x1+1,x2,xn+1), (x1+1,x2+1,xn-1), (x1+1,x2+1,xn), (x1+1,x2+1,xn+1)}
- Over R
  - vn(x1,..,xn)=returns the open set centered in the point x1,x2,x3 for the topology that defines the vicinity, $B(x,r) = \{y \in \Re^m / d(x,y) < r\}$

Nucleus function example

- Over Z
- nc(x1,..,xn) = {(x1,..,xn)}
- Over R
- nc(x1,..,xn)=returns the open set centered in the point x1,x2,x3 for the topology that defines the nucleus.

Using SDL

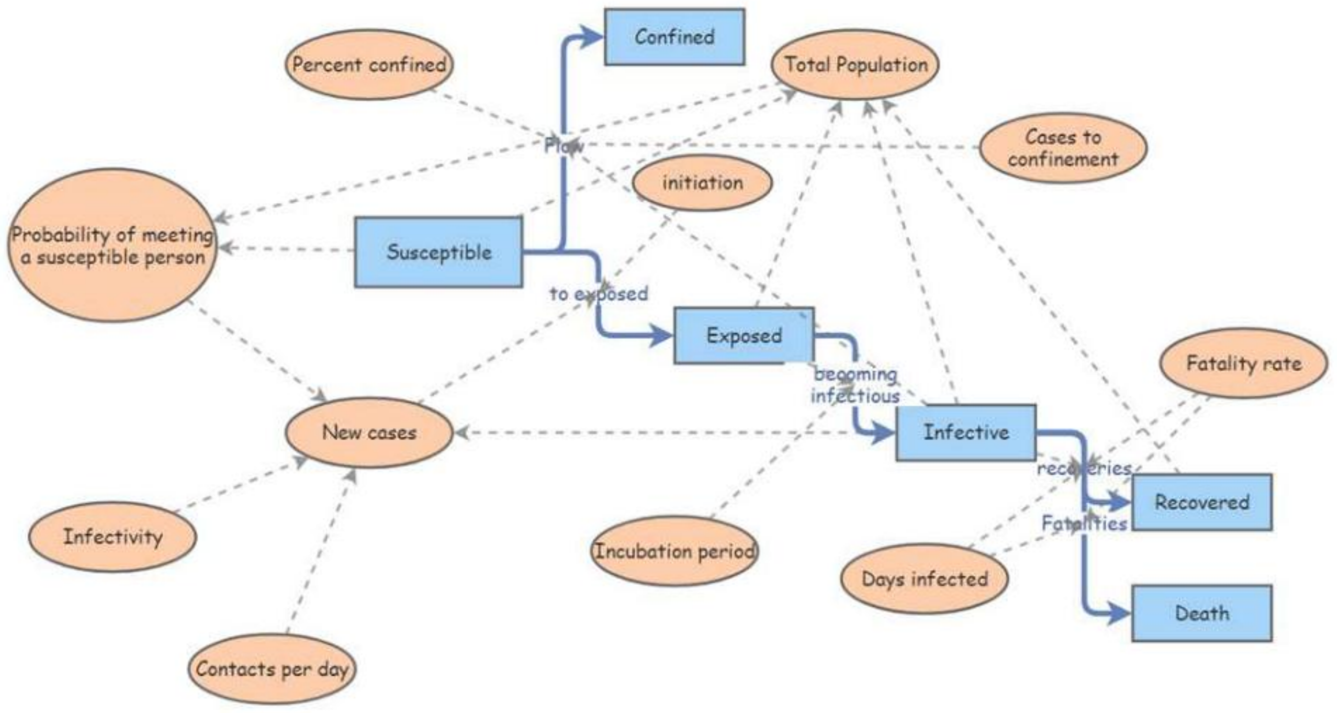
Defining CA models with SDL

Alternatives

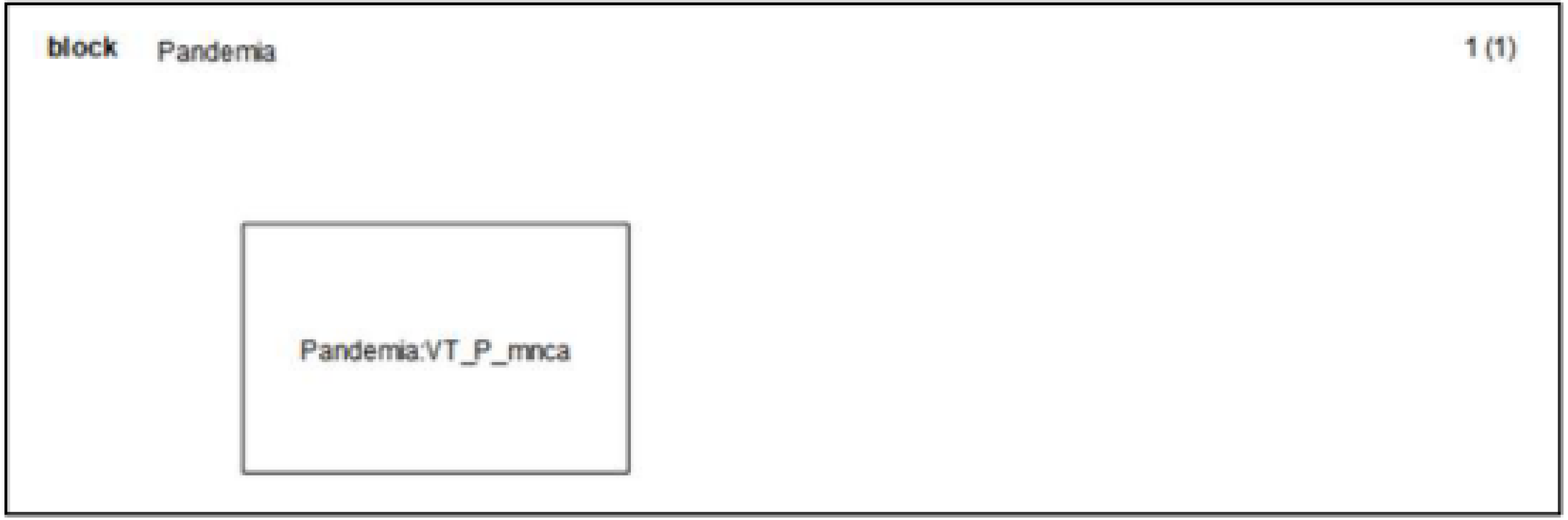
- CELL-DEVS, SDL
- Both are equivalent. Also, we can use Petri Nets, see
  - I. Barragán, J. C. Seck-Tuoh, and J. Medina, *Relationship between Petri Nets and Cellular Automata for the Analysis of Flexible Manufacturing Systems*, in *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Vol. 7630 LNAI (2013), pp. 338–349.

Justification

- Need to simplify the validation process of the model.
- Need to replicate the behavior faster.



System

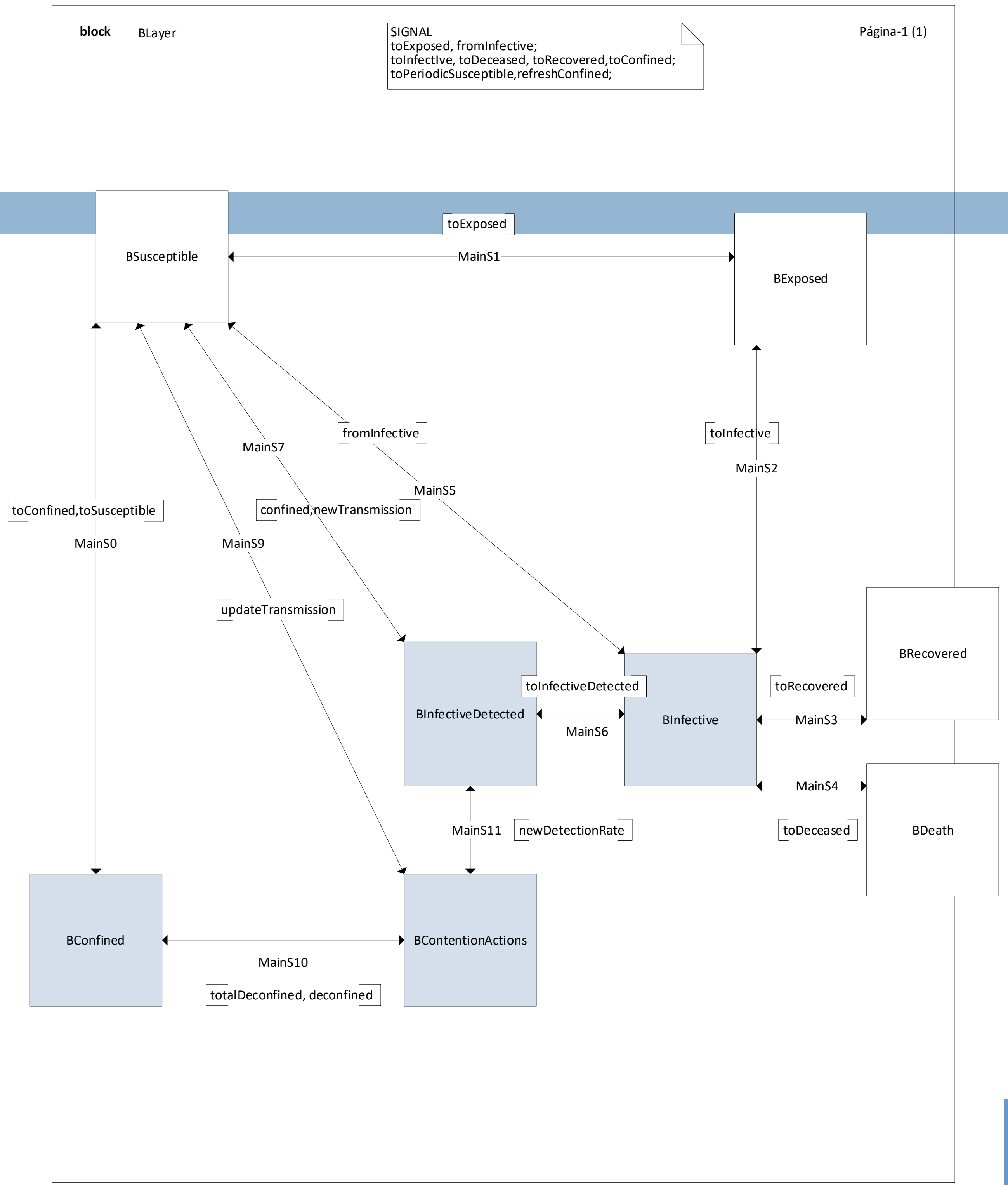


block MNCA_Pandemia inherits VT_P_mnca

BLayer

DCL

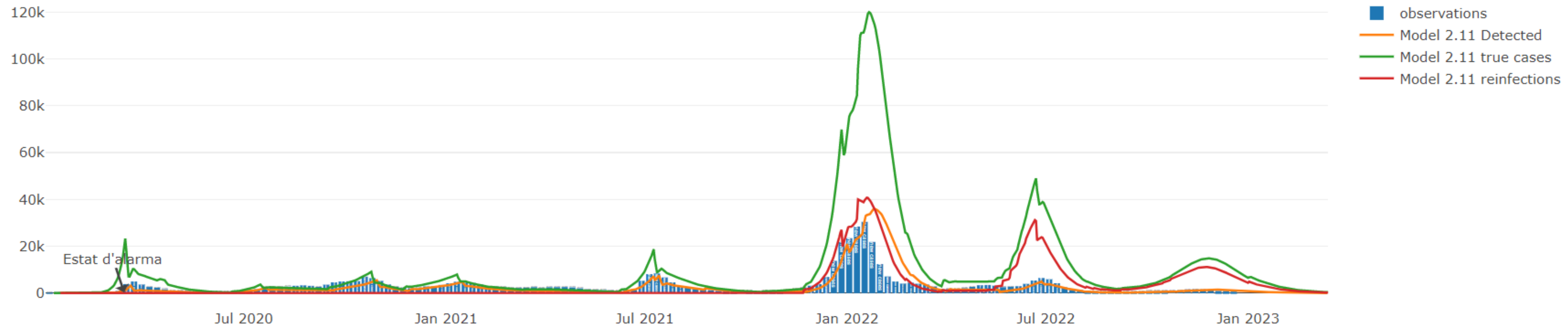
```
double layer_main='Initialize.img';
double layer_population='Population.img';
double layer_initial_infective='Initial_infective.img';
double layer_infective='Infective.img';
double layer_susceptible='Susceptible.img';
double layer_exposed='Exposed.img';
double layer_recovered='Recovered.img';
double layer_deceased='Deceased.img';
double layer_confined='Confined.img';
double layer_new_exposed='New_exposed.img';
double layer_new_infective='New_infective.img';
double layer_infective_detected='Infective_detected.img';
double layer_transmision='Transmision.img';
double layer_pdetected='pDetected.img';
double layer_confinement='Confinement.img';
double layer_new_confined_cases='New_confined_cases.img';
```


block BLayer
SIGNAL
toExposed, fromInfective;
toInfectIve, toDeceased, toRecovered,toConfined;
toPeriodicSusceptible,refreshConfined;
Página-1 (1)
BSusceptible
toExposed
MainS1
BExposed
fromInfective
toInfective
MainS2
MainS7
MainS5
toConfined,toSusceptible
confined,newTransmission
MainS0
MainS9
updateTransmission
BRecovered
BInfectiveDetected
toInfectiveDetected
BInfective
toRecovered
MainS6
MainS3
MainS4
MainS11
newDetectionRate
toDeceased
BDeath
BConfined
BContentionActions
MainS10
totalDeconfined, deconfined

SDL-PAND KPIs Model actual

New cases | R value (Digital Shadow) | Accumulated curve (Digital Shadow)

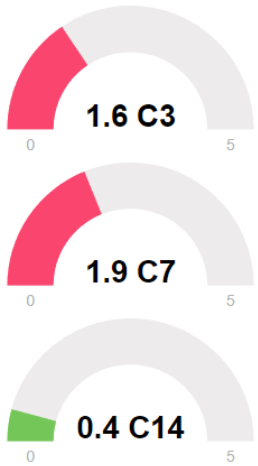
The simulation model allows to have a forecast of the evolution of the pandemic. It is corrected based on the actual data of the evolution of the pandemic. The current model is model 2.10. The R is calculated with the EpiEstim package of the statistical language "R".



KPI details

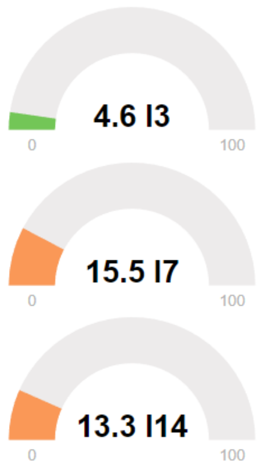
C3: 3 days moving average; C7: 7 days moving average; C14: 14 days moving average. C7 and C14 are more stable than C3, picking up trend variations in full week. Especially C3 has bias, an increase (very unstable) may represent a quick warning of possible worsening. Calculated for 100,000 inhabitants.

Effective growth. Digital Shadow



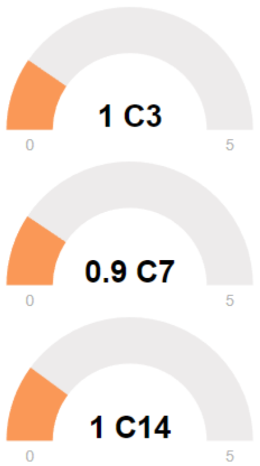
Detected cases.

Incidence. Digital Shadow.



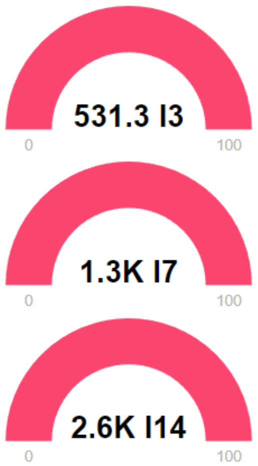
Detected cases.

Effective growth. Digital Twin



True cases.

Incidence. Digital Twin.



True cases.

Slap avalanche model

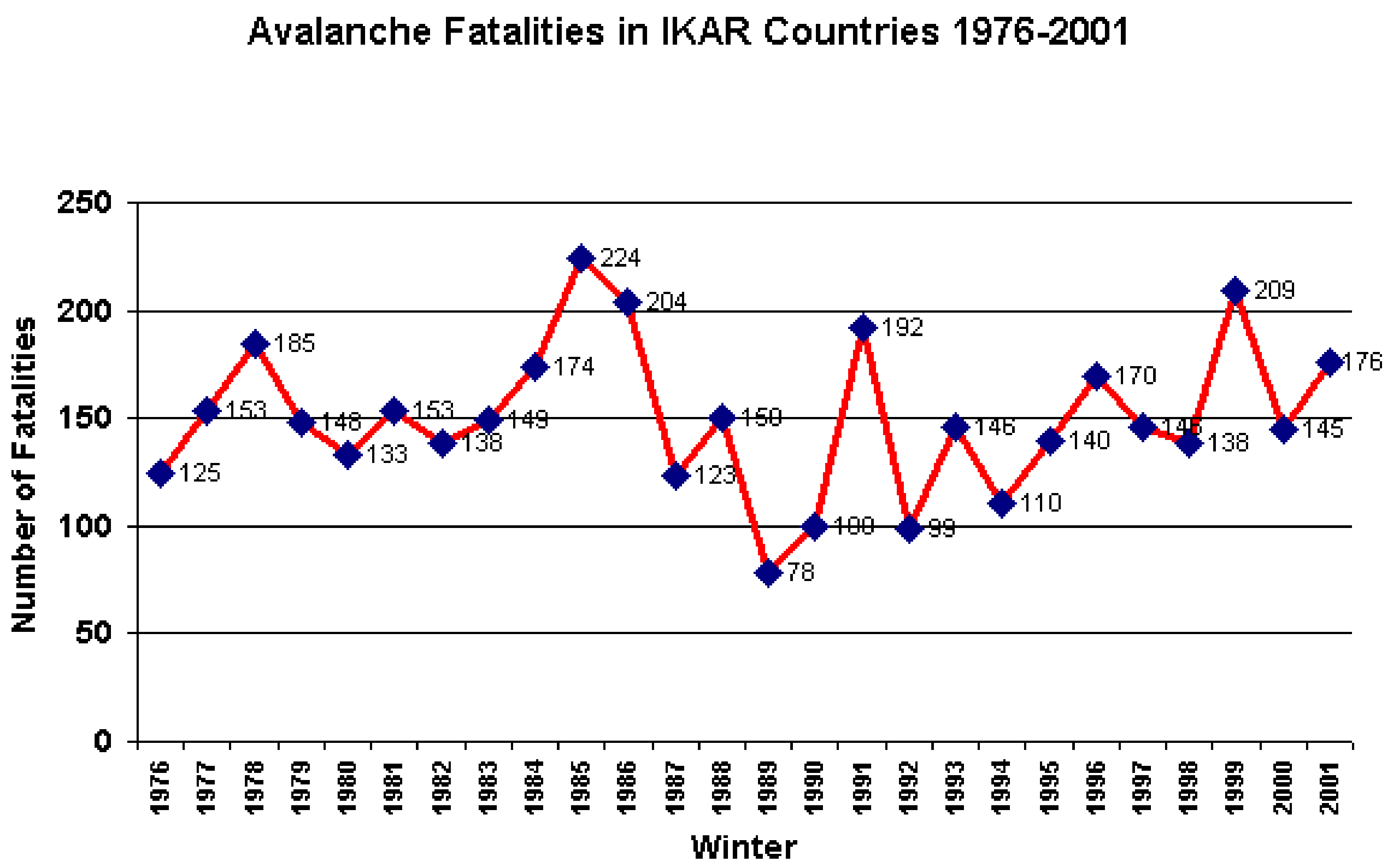
Slap avalanche model

Avalanche

Two main types of **snow** avalanche:

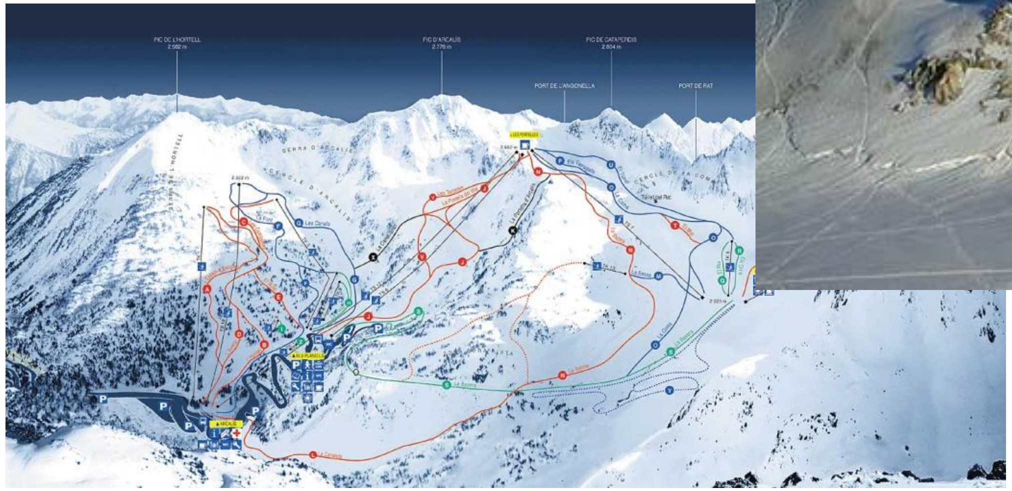
- **Loose-snow** avalanche originates at a point and propagates downhill by successively dislodging increasing numbers of poorly cohering snow grains, typically gaining width as movement continues down slope.
- **Slab avalanche,** occurs when a distinct cohesive snow layer breaks away as a unit and slides because it is poorly anchored to the snow or ground below

Avalanche fatalities in IKAR Countries



UNIVERSITAT POLITÈCNICA DE CATALUNYA

Some photos



UNIVERSITAT POLITÈCNICA DE CATALUNYA

Avalanche Model data

Name	Type	Description	Qtt	Source	Modifiable
Height	Raster	Layer representing the height of the environment.	1	ICC	No
Thickness of the snow	Raster	Represents the thickness of the “slab snow”	1	Meteocat	Yes
Floor features	Raster	Represents the kind of surface (rocks, sand, snow, ice,..). Each surface has his own specific rough parameter.	1	Meteocat Creaf	No
Snow that causes the slab features	Raster	Density, compactness of the snow.	1	Meteocat	Yes*
Obstacles	Raster	Represents the obstacles that have the environment (small rocks, big rocks, houses, trees,…)	N	Creaf	Yes
Crack	Vectorial	Line representing the breakdown of the ice.	1	Input data	Yes, at beginning.
State of the snow	Raster	Shows the state of the terrain, **empty**, **static** and **dynamic**	1	Meteocat	Yes

Avalanche Model

- $6+N{:}2\text{-}AC^{4+N}$ on Z^2

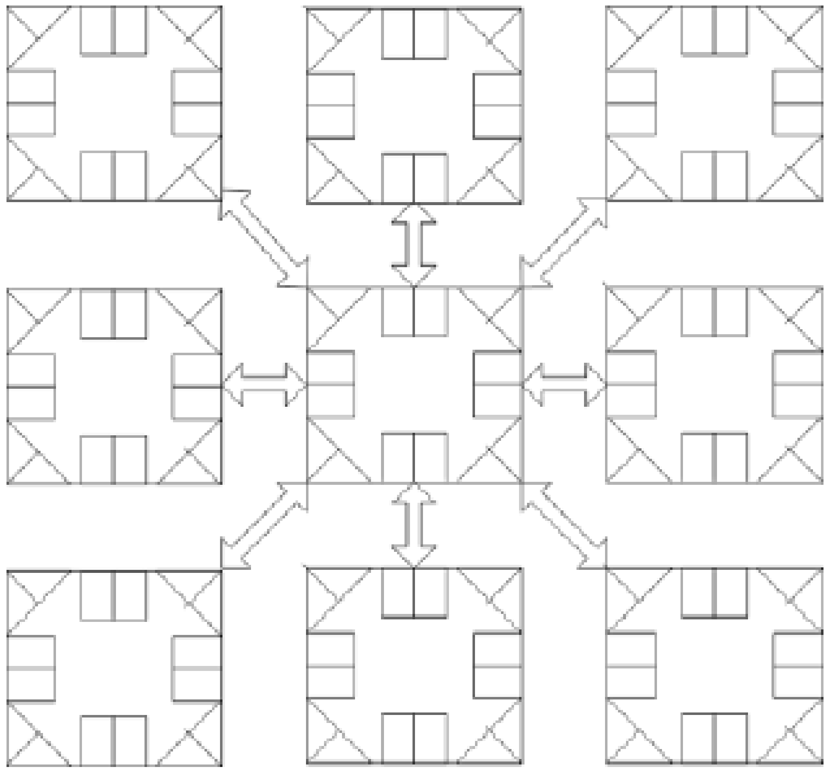
Vicinity and nucleus function

- Vicinity function: vn(x1,x1) = {(x1-1,x2-1), (x1-1,x2), (x1-1,x2+1), (x1,x2-1), (x1,x2), (x1,x2+1), (x1+1,x2-1), (x1+1,x2), (x1+1,x2+1)}
- Nucleus function: nc(x1,x1)= {(x1,x1)}

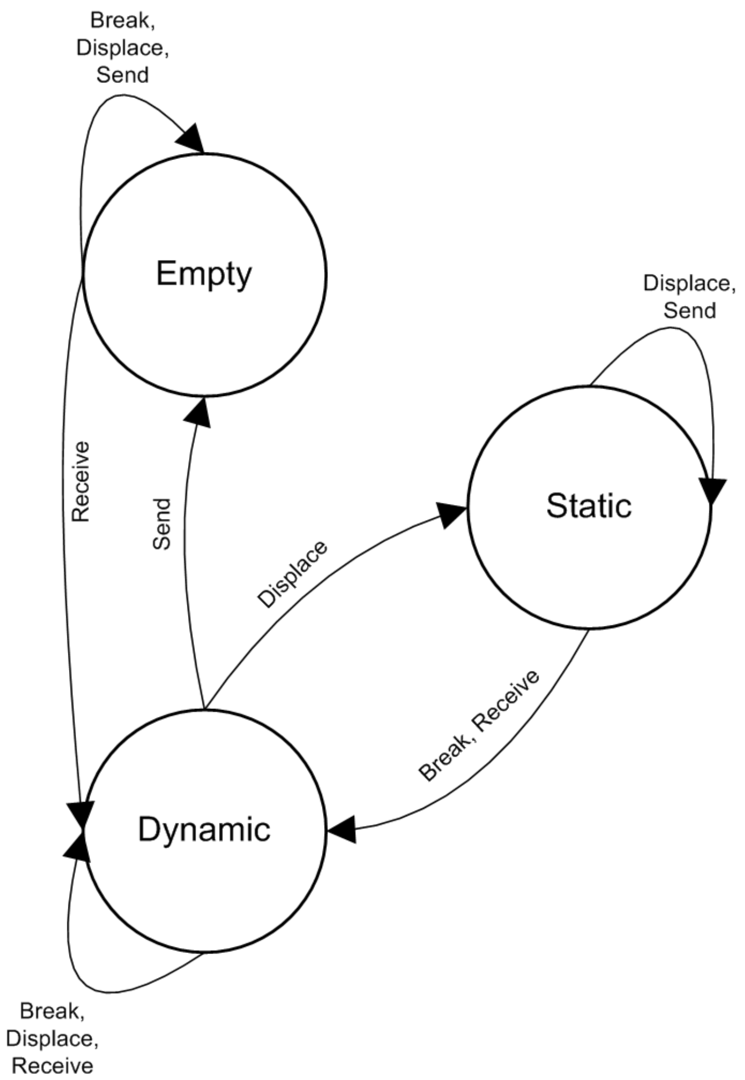
Evolution functions

- $E_2[i]$: Thickness of the snow. The function that rules this layer is "Modify information(p)"
- $E_4[i]$: Density, compactness of the snow, in our case is 0.5 (Mears 1976).
- $E_6[i]$: State of the snow. The function is defined in the next diagrams.
- $E_N[i]$: Obstacles. The function that defines the obstacles we use in the model.

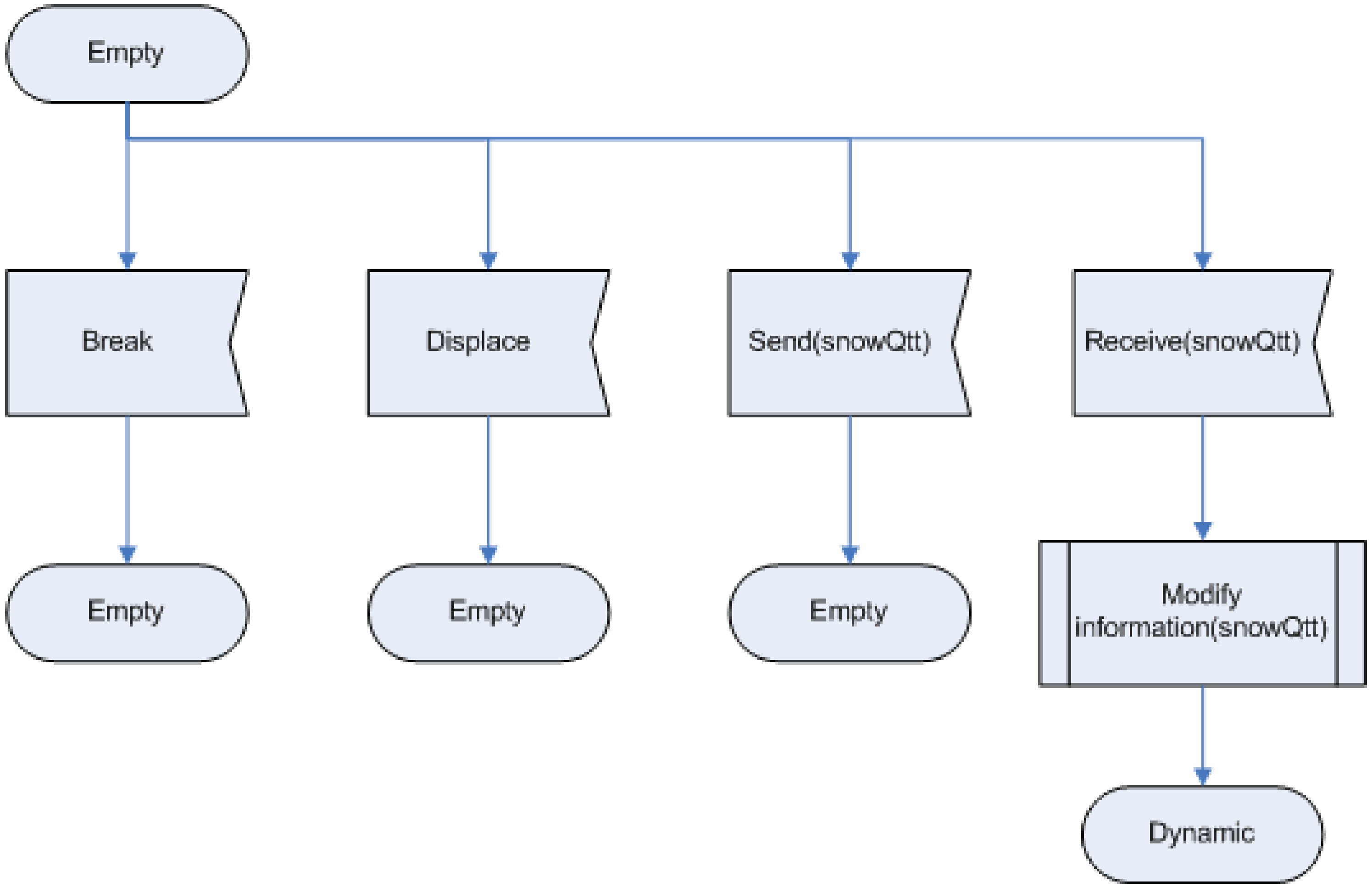
Moore neighbourhood



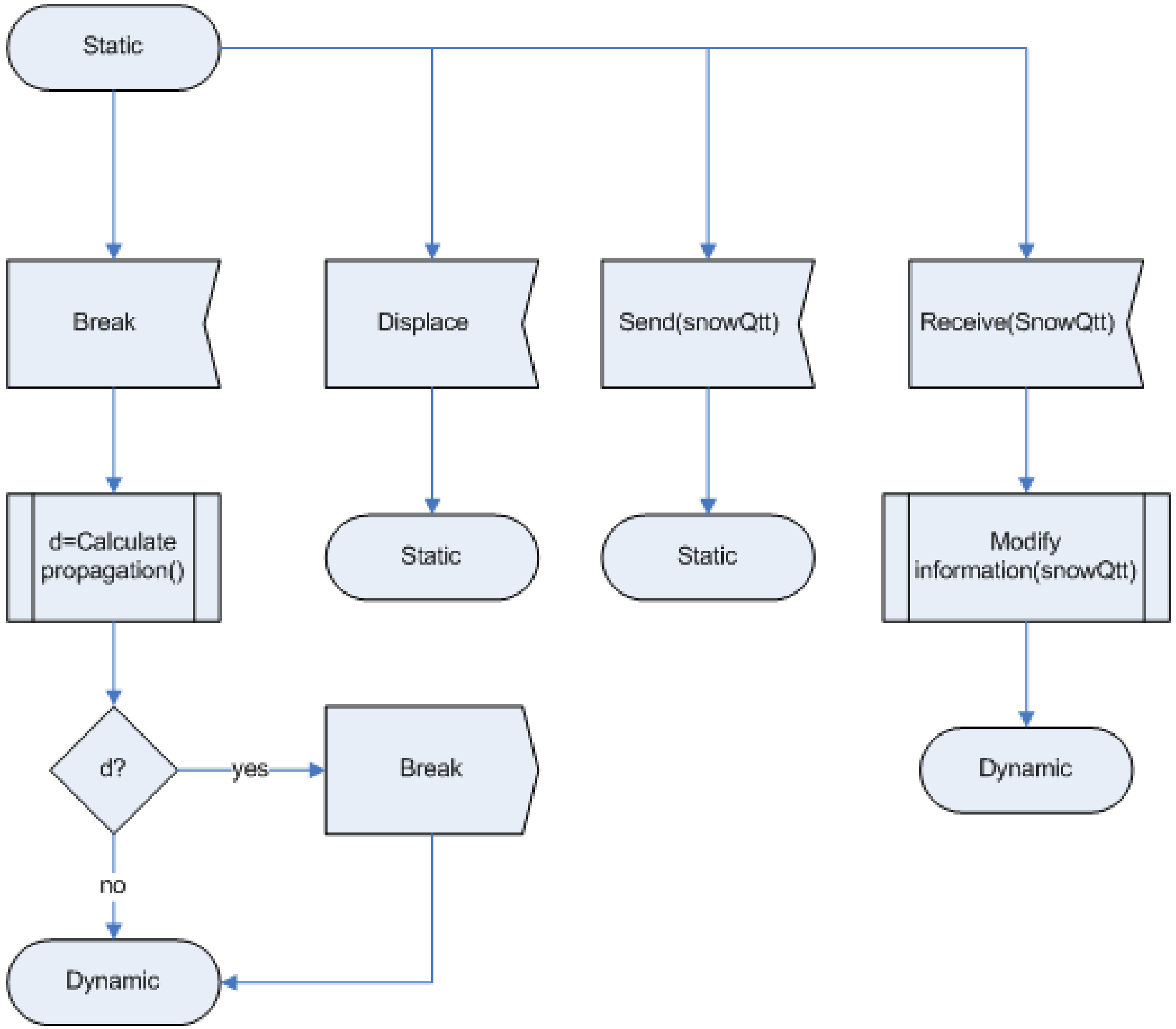
Λ:state of the snow



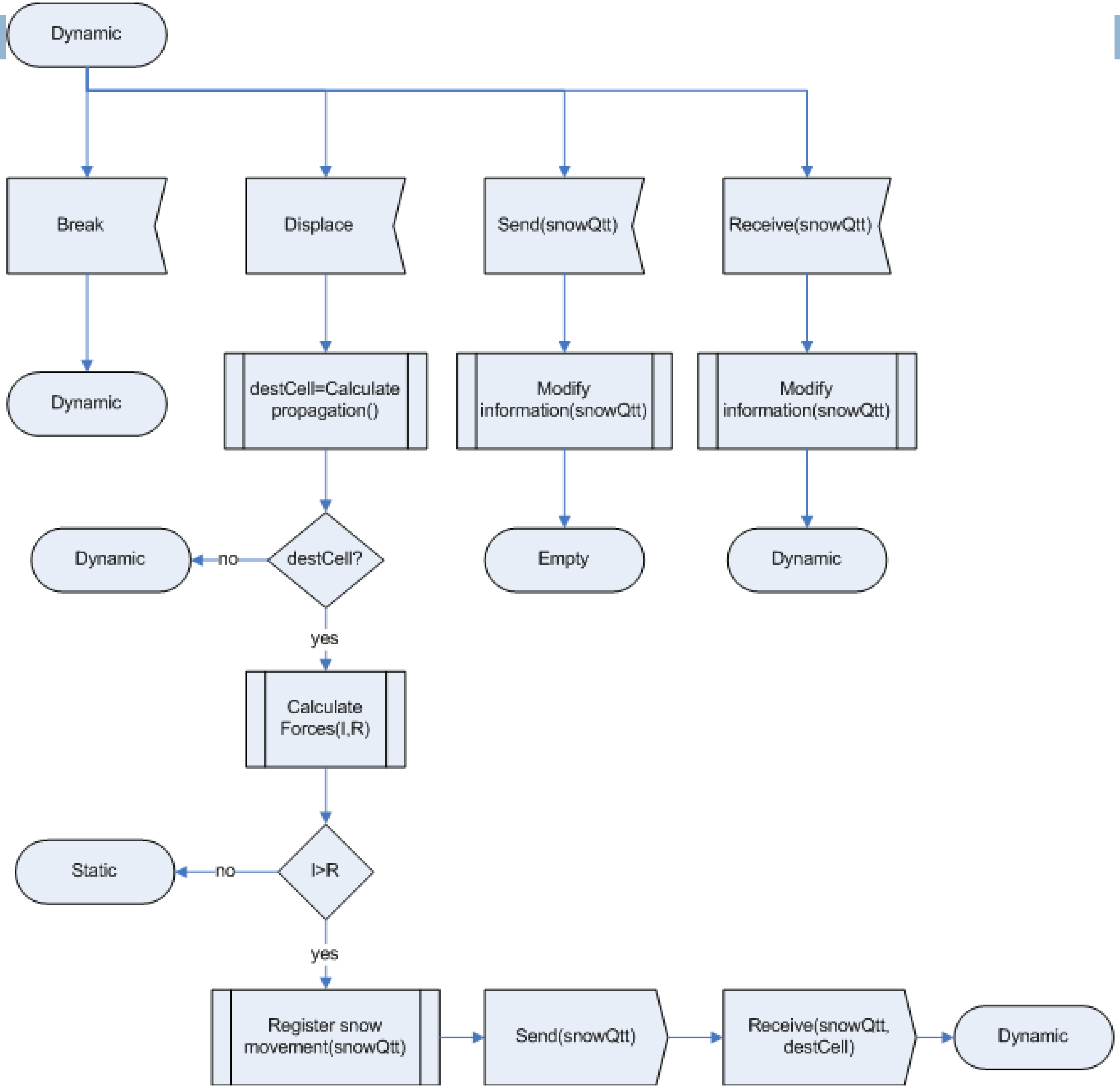
Empty process



Static process



Dynamic process



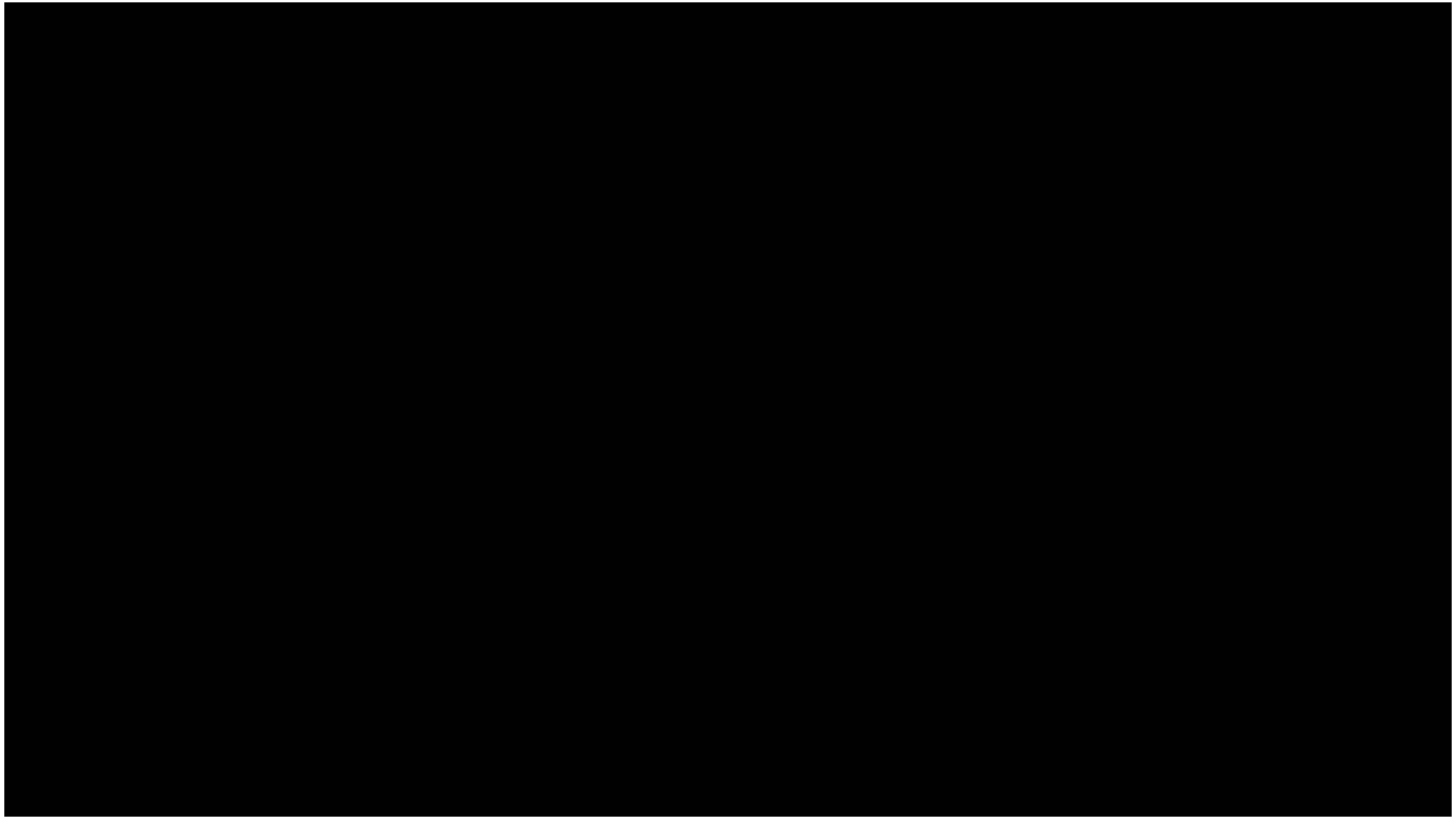
Evolution function

- The increment in the force is used in the next expression to determine if the snow continues its movement to other cell, or stops its movement, if the force is equal to zero.
- $F_{i,t} = \max(IF_{i,t} + \Delta F_{i,t}, 0)$

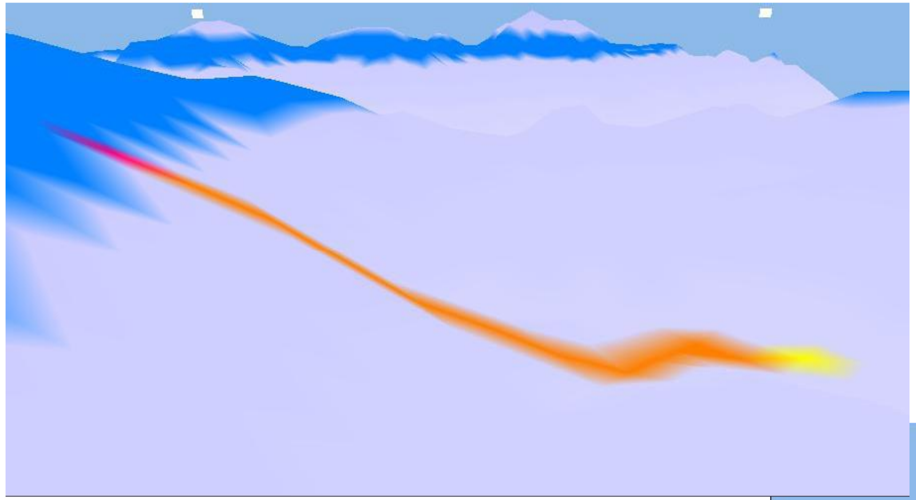
Evolution function

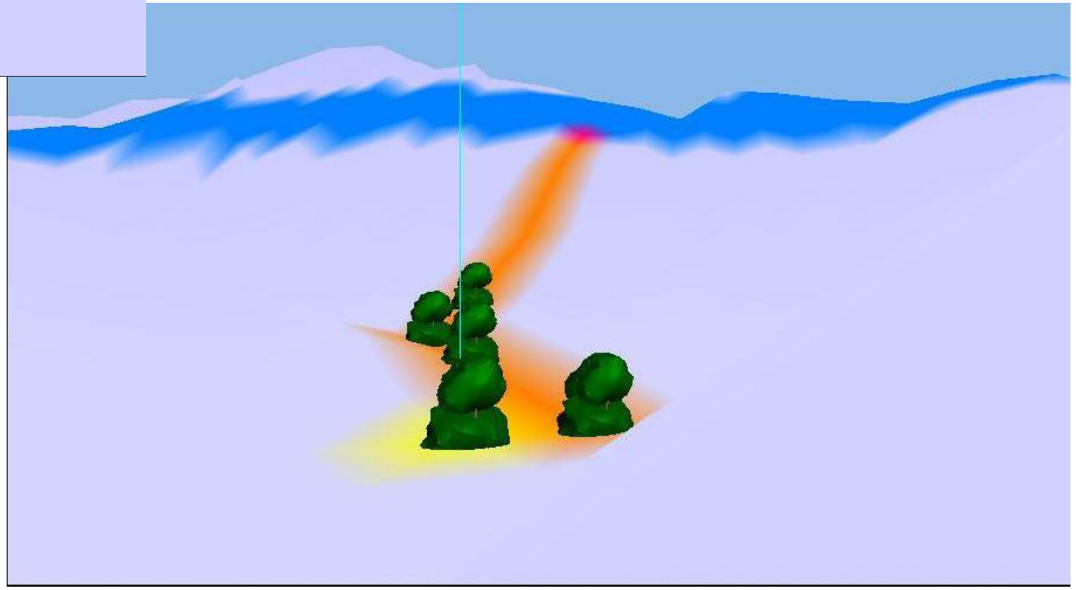
- $IF_{i,t}$=Impulse force, depends on the quantity and quality of the snow, and the slope.
- $SFF_{i,t}$= Sliding friction force between the avalanche and the underlying snow or ground.
- $IFF_{i,t}$= Internal dynamic shear resistance due to collisions and momentum exchange between particles and blocks of snow, (internal friction force).
- $ASFF_{i,t}$= Turbulent friction within the snow/air suspension, (air suspension friction force).
- $AFF_{i,t}$=Shear between the avalanche and the surrounding air, (air friction force).
- $FFF_{i,t}$= Fluid-dynamic drag at the front of the avalanche (front friction force).
- $OFF_{i,t}$=Obstacle friction force.
  - $\Delta F_{i,t} = IF_{i,t} - (SFF_{i,t} + IFF_{i,t} + ASFF_{i,t} + AFF_{i,t} + FFF_{i,t} + OFF_{i,t})$

Slap avalanche

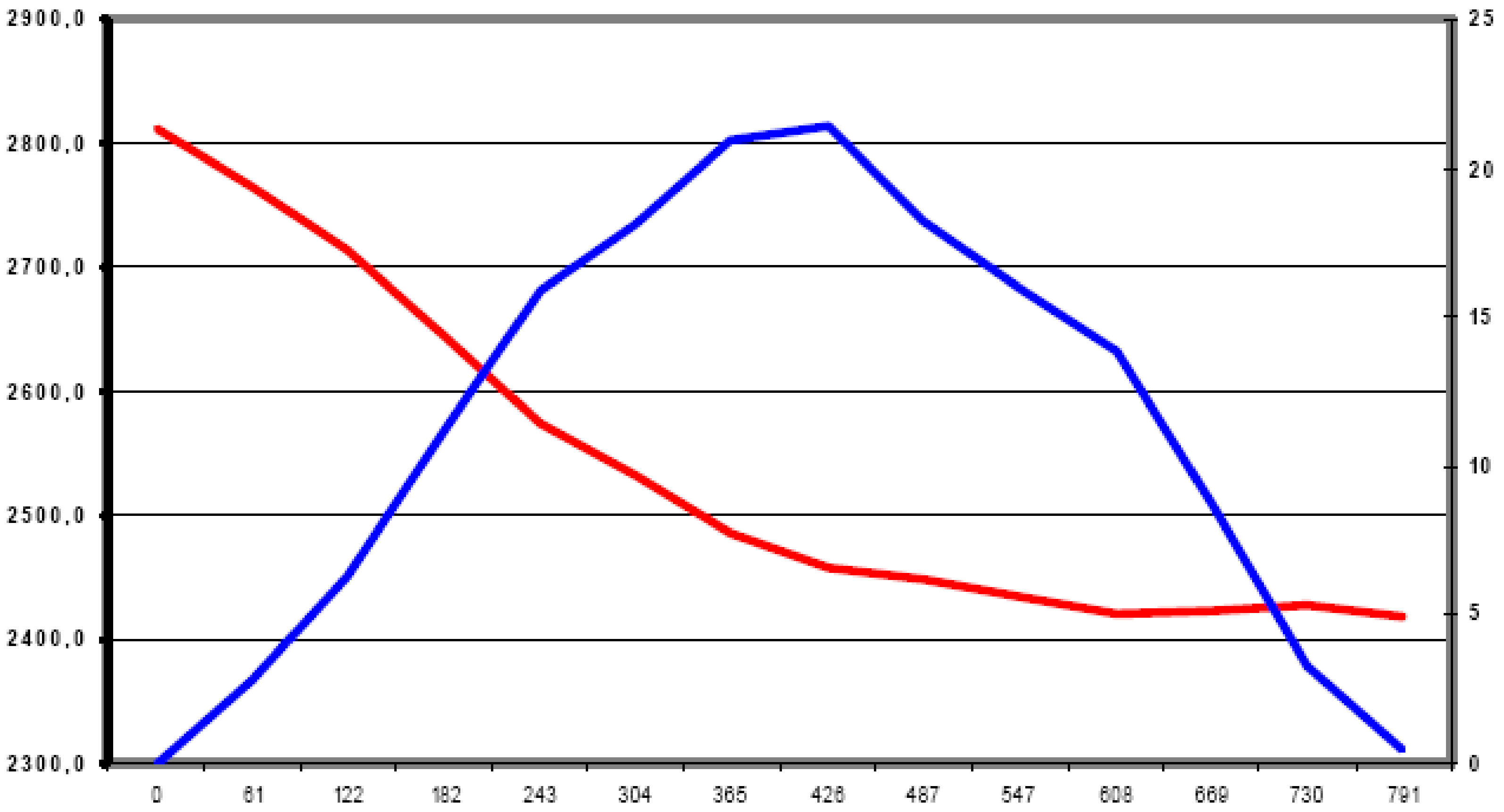


Results





Results



2900,0
2800,0
2700,0
2600,0
2500,0
2400,0
2300,0
25
20
15
10
5
0
0
61
122
182
243
304
365
426
487
547
608
669
730
791

Results (5)

Desencadenant:

Localització: 8 cel·les, de (108, 33) fins (115, 33)

Gruix de neu de placa: 50cm (per totes les cel·les fracturades)

Terreny subjacent: Neu dura

Obstacles: No

Característiques de l'allau:

Terreny subjacent del camí: Neu dura

Màxima distancia recorreguda: 1101,14m

Desnivell superat: 520,40m

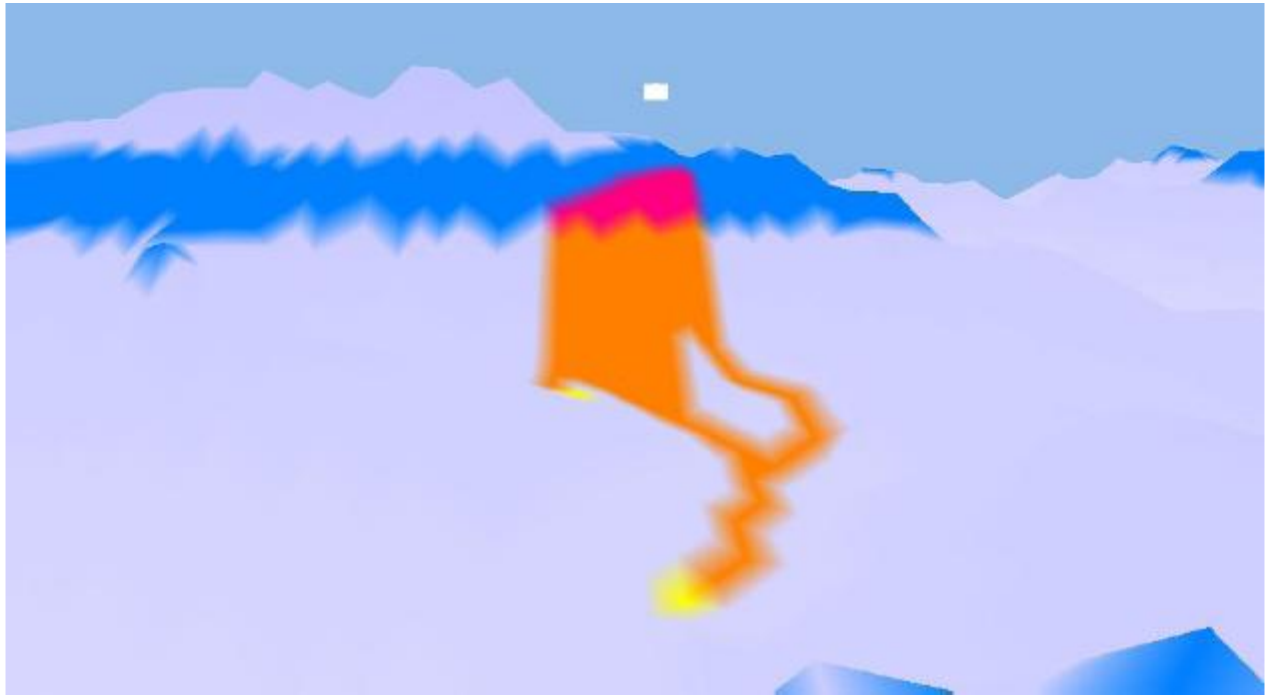
Massa transportada: 10625kg

Massa de neu en dipòsit: 9957,50kg

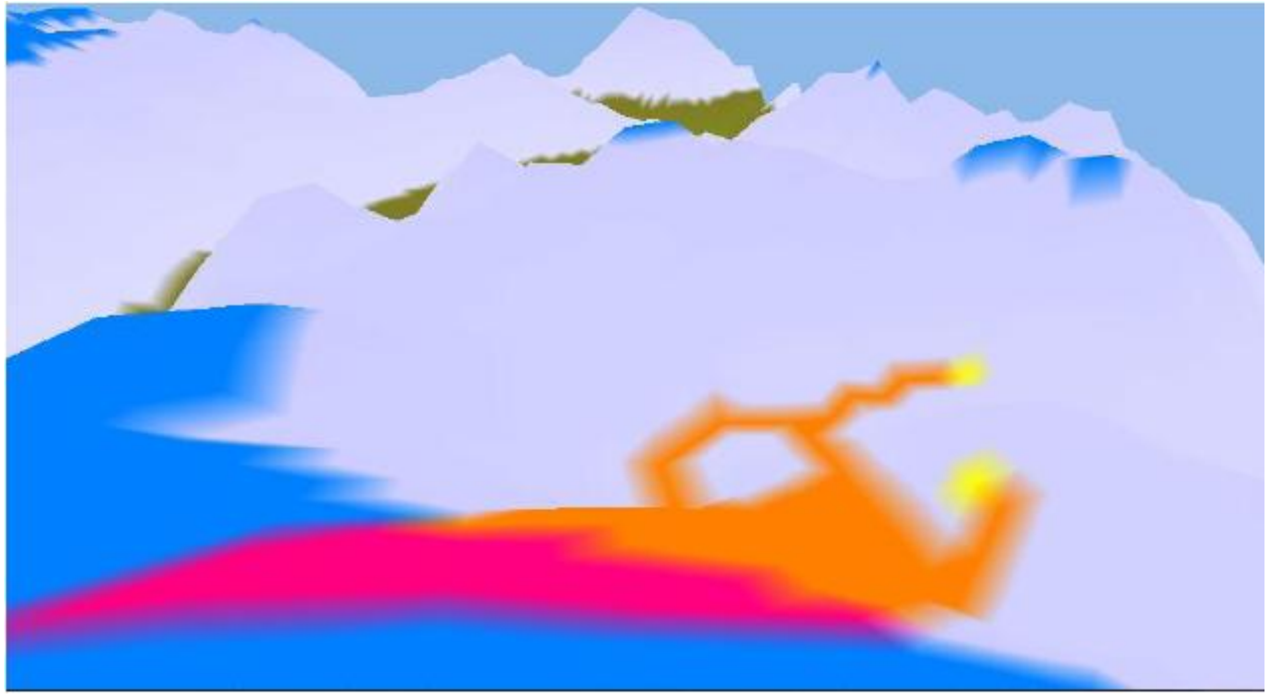
Massa de neu perduda pel camí: 667,5kg

Velocitat màxima: 67,23m/s

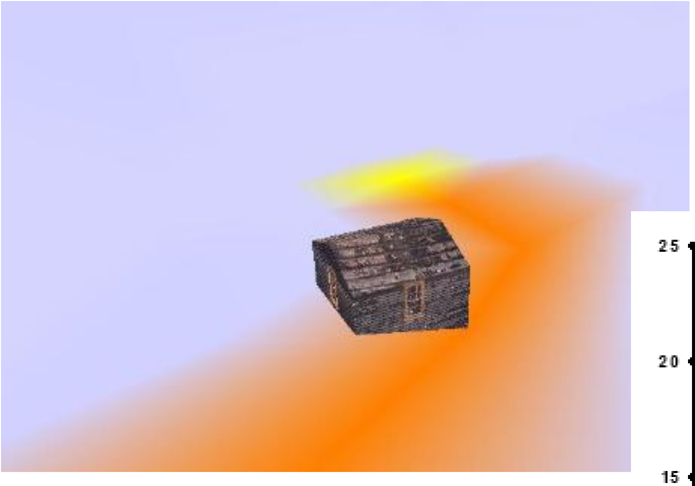
Results (5)



Results (5)



Results (1 vs 3)



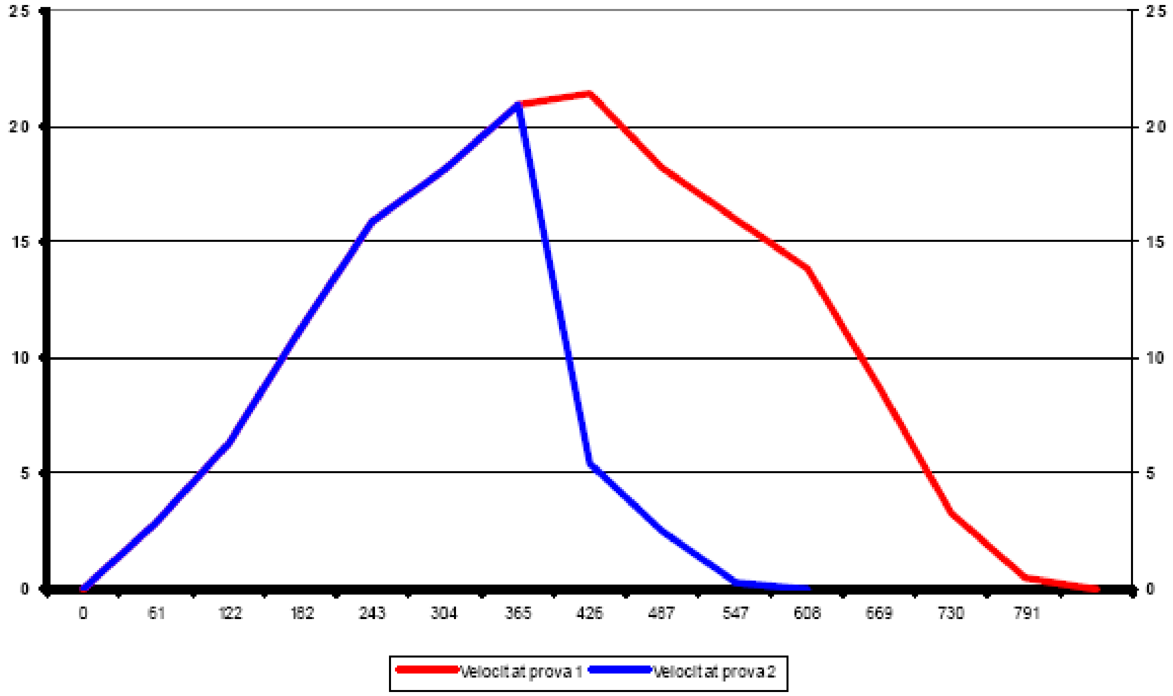
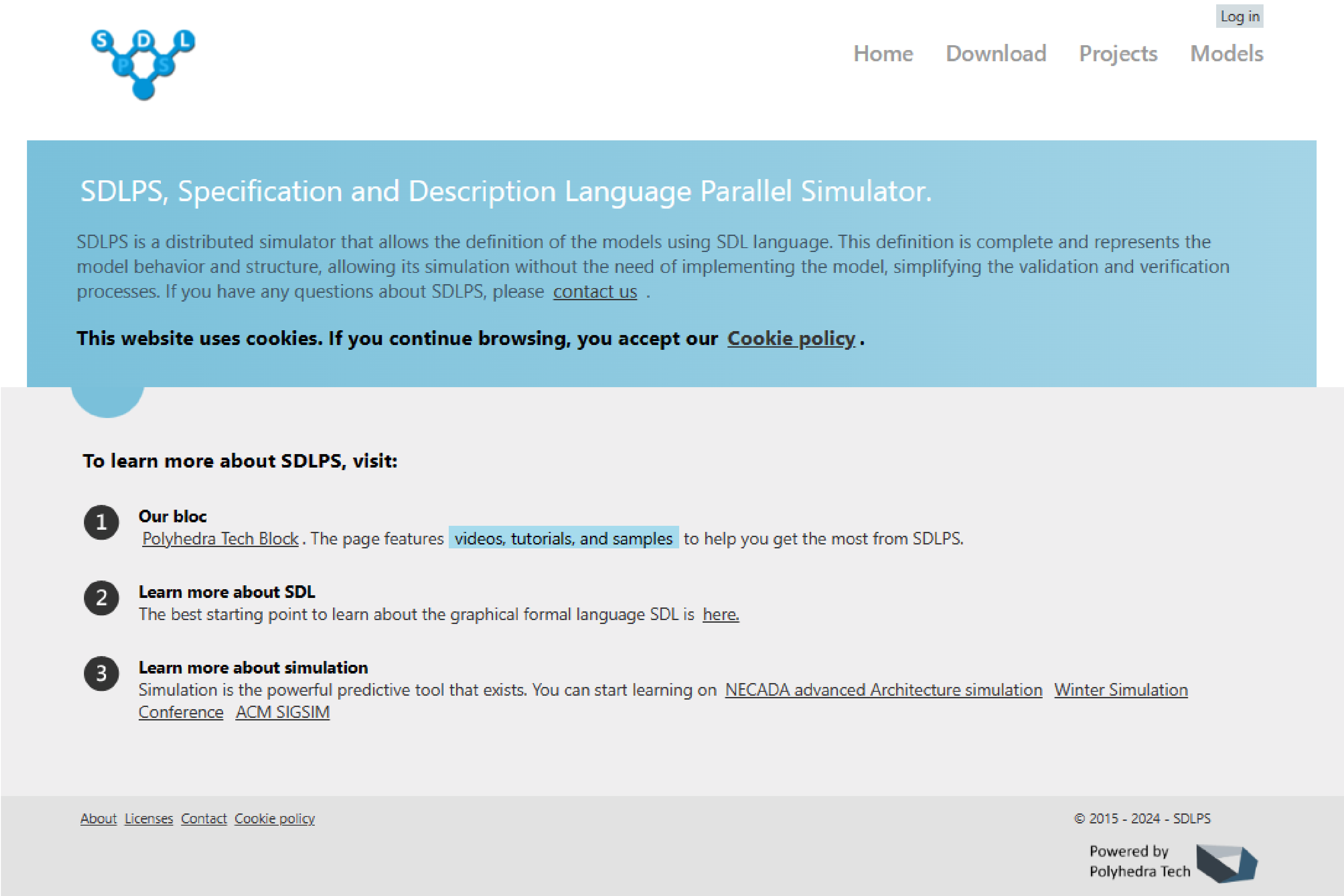


Figura 7.9 – Velocitat Sim.1 VS Velocitat Sim.3

Some tools

- SDLPS



More tools

- Play John Conway's Game of Life (playgameoflife.com)
- Cellular Automata Sim by AlexMulkerrin (itch.io)
- Cellarium by Benjamin Mastripolito (itch.io)
- Cellular Automaton Explorer - Wolfram Demonstrations Project
- Automata Ecosystem - Cellular Automata Simulation en Steam (steampowered.com)

Some references

- Doran, Jim. E. 2000, *HARD PROBLEMS IN THE USE OF AGENT-BASED MODELLING,* Proceedings of the Fifth International Conference on Logic and Methodology, Cologne, October 3-6
- Roher, G. 2000, *Can Simulation models Support Social Research? A Critical Discussion.*
- Itzhak Benenson, Itzhak Omer. 2001, *Agent-Based modeling of residential Distribution.* Departament of Geography and human Environment University Tel Aviv. Israel.
- Epstein, Joshua M. Axtell, Robert. 1996 *Growing Artificial Societies: Social Science from the Bottom Up.* MIT Press, Cambridge, MA, USA.

UNIVERSITAT POLITÈCNICA DE CATALUNYA

Some references

- Batty, Michael, 2005, *Cities and complexity*.The MIT Press.
- Torrens, Paul M. Benenson, Itzhak. 2004, Geosimulation, John Wiley & Sons Ltd., ISBN: 0-470-84349-7
- Wuensche, Andrew; Lesser, Mike. 1992, *The gobal dynamics of cellular automata*, Santa Fe institute, studies in the sciences of complexity, reference volume I, Addison Wesley, 1992
- G. 't Hooft, *The Cellular Automaton Interpretation of Quantum Mechanics*, **2015**, (2014).

Some references

- Fonseca i Casas, Pau; Casanovas, Josep; Montero, Jordi. 2004c. *Cellular automata and intelligent agents used to model natural disasters with discrete simulation*. Proceedings of EMS 2004.
- Fonseca P.; Casanovas J. 2005. ESS205, *Simplifying GIS data use inside discrete event simulation model through m:n-AC cellular automaton*; Proceedings ESS 2005.
- Fonseca i Casas P., Rodríguez Fontoba,S., 2007, Using GIS data in a m:n-ACk cellular automaton to perform an avalanche simulation. Geographical Information Science Research UK Conference 2007. National University of Ireland Maynooth. http://ncg.nuim.ie/gisruk/materials/proceedings/
- P. Fonseca i Casas, M. Colls, and J. Casanovas, A Novel Model to Predict a Slab Avalanche Configuration Using M : N-CA k Cellular Automata, Comput. Environ. Urban Syst. 35, 12 (2011).

More references

- P. Fonseca i Casas, Towards a Representation of Cellular Automaton Using Specification and Description Language, in System Analysis and Modeling. Languages, Methods, and Tools for Industry 4.0, edited by P. Fonseca i Casas, M.-R. Sancho, and E. Sherratt (Springer, 2019), pp. 163–179.
- P. Fonseca i Casas, J. Garcia i Subirana, V. Garcia i Carrasco, J. L. Silva de Barcellos, J. Roma, and X. Pi, SDL Cellular Automaton COVID-19 Conceptualization, in Proceedings of the 12th System Analysis and Modelling Conference (ACM, New York, NY, USA, 2020), pp. 144–153.
- Fonseca i Casas, Pau, The Multi-n-Dimensional Cellular Automaton: A Unified Framework for Tensorial, Discrete, and Continuous Simulations—A Computational Definition of Time, Complexity, 2025, 3088010, 35 pages, 2025. https://doi.org/10.1155/cplx/3088010

UNIVERSITAT POLITÈCNICA DE CATALUNYA